

# PreOne Enterprise

# Frontend Architecture

Frontend Architecture — Enterprise Preschool Operating System

Document Version: 1.0

Status: Frontend Architecture Freeze

Framework: Next.js 16 + React 19 + TypeScript

Date: 2026-07-14

Product: PreOne — Enterprise Preschool Operating System

Predecessor: Vision v1.0, Master PRD v1.0, DDD v1.0, ADR Catalog, UI Design Philosophy, API Contract Catalog, Backend Technical Design

Successor: Web Portal Implementation, Component Library, Storybook, Playwright Tests, Production Build

Classification: Internal Engineering Reference

Prepared by: PreOne Frontend Architecture & Engineering Team

# Table of Contents

|                                             |           |
|---------------------------------------------|-----------|
| <b>1. Frontend Overview.....</b>            | <b>1</b>  |
| 1.1 Purpose.....                            | 1         |
| 1.2 Scope.....                              | 2         |
| 1.3 Audience.....                           | 2         |
| 1.4 Document Conventions.....               | 4         |
| 1.5 Related Documents.....                  | 5         |
| <b>2. Technology Stack.....</b>             | <b>5</b>  |
| 2.1 Stack Pipeline.....                     | 5         |
| 2.2 Component Catalog.....                  | 8         |
| 2.3 Version Pinning Strategy.....           | 8         |
| 2.4 Type Sharing with Backend.....          | 9         |
| <b>3. Project Structure.....</b>            | <b>10</b> |
| 3.1 Top-Level Layout.....                   | 11        |
| 3.2 Top-Level Directory Roles.....          | 13        |
| 3.3 Monorepo Layout.....                    | 13        |
| <b>4. App Router.....</b>                   | <b>15</b> |
| 4.1 Route Groups.....                       | 15        |
| 4.2 App Router Features.....                | 19        |
| 4.3 Route Metadata.....                     | 20        |
| <b>5. Module Structure.....</b>             | <b>23</b> |
| 5.1 Module Template (student/ example)..... | 23        |
| 5.2 Directory Responsibilities.....         | 25        |
| 5.3 Module List.....                        | 25        |
| 5.4 Module Barrel Pattern.....              | 25        |

|                                          |           |
|------------------------------------------|-----------|
| <b>6. Layout System.....</b>             | <b>27</b> |
| 6.1 Layout Catalog.....                  | 27        |
| 6.2 Layout Slots.....                    | 27        |
| 6.3 Layout Composition.....              | 28        |
| <b>7. State Management.....</b>          | <b>30</b> |
| 7.1 Global State (Zustand).....          | 30        |
| 7.2 Server State (TanStack Query).....   | 31        |
| 7.3 State Management Rules.....          | 31        |
| 7.4 Zustand Store Example.....           | 32        |
| <b>8. API Layer.....</b>                 | <b>36</b> |
| 8.1 API Layer Flow.....                  | 37        |
| 8.2 API Service Catalog.....             | 39        |
| 8.3 Axios Interceptor Rules.....         | 39        |
| 8.4 Service Example.....                 | 41        |
| <b>9. Authentication.....</b>            | <b>44</b> |
| 9.1 Authentication Flow.....             | 45        |
| 9.2 Authentication Features.....         | 47        |
| 9.3 Token Storage Strategy.....          | 48        |
| 9.4 Multi-School Context Switch.....     | 48        |
| <b>10. RBAC.....</b>                     | <b>50</b> |
| 10.1 RBAC Layers.....                    | 50        |
| 10.2 RBAC Examples.....                  | 50        |
| 10.3 RBAC Rules.....                     | 50        |
| 10.4 <Can> Component Implementation..... | 52        |
| <b>11. Navigation.....</b>               | <b>57</b> |
| 11.1 Navigation Types.....               | 57        |
| 11.2 Navigation Rules.....               | 57        |
| 11.3 Command Palette (Cmd+K).....        | 58        |

|                                       |           |
|---------------------------------------|-----------|
| <b>12. Forms.....</b>                 | <b>61</b> |
| 12.1 Form Patterns.....               | 61        |
| 12.2 Form Submission Flow.....        | 61        |
| 12.3 Form Rules.....                  | 64        |
| 12.4 Example — CreateStudentForm..... | 65        |
| <b>13. Validation.....</b>            | <b>72</b> |
| 13.1 Validation Levels.....           | 72        |
| 13.2 Validation Examples.....         | 72        |
| 13.3 Zod Schema Example.....          | 72        |
| <b>14. Theme Engine.....</b>          | <b>75</b> |
| 14.1 Theme Hierarchy.....             | 75        |
| 14.2 Theme Hierarchy Layers.....      | 77        |
| 14.3 Theme Tokens.....                | 77        |
| 14.4 Theme Modes.....                 | 77        |
| 14.5 Theme Application.....           | 77        |
| <b>15. Component Library.....</b>     | <b>81</b> |
| 15.1 Base Components.....             | 81        |
| 15.2 Composite Components.....        | 81        |
| 15.3 Component Rules.....             | 81        |
| 15.4 Storybook Integration.....       | 83        |
| <b>16. Data Tables.....</b>           | <b>87</b> |
| 16.1 Table Features.....              | 87        |
| 16.2 Table Rules.....                 | 87        |
| 16.3 Table Component Example.....     | 88        |
| <b>17. Charts.....</b>                | <b>94</b> |
| 17.1 Chart Types.....                 | 94        |
| 17.2 Chart Rules.....                 | 94        |
| 17.3 Chart Example.....               | 96        |

|                                        |            |
|----------------------------------------|------------|
| <b>18. Notifications.....</b>          | <b>100</b> |
| 18.1 Notification Types.....           | 100        |
| 18.2 Notification Rules.....           | 100        |
| 18.3 Toast System.....                 | 101        |
| <b>19. File Upload.....</b>            | <b>105</b> |
| 19.1 Upload Features.....              | 105        |
| 19.2 Upload Flow.....                  | 105        |
| 19.3 FileUpload Component Example..... | 107        |
| <b>20. Error Handling.....</b>         | <b>113</b> |
| 20.1 Error Types.....                  | 113        |
| 20.2 Error Flow.....                   | 113        |
| 20.3 Error Handling Rules.....         | 116        |
| 20.4 Global Error Boundary.....        | 117        |
| <b>21. Performance.....</b>            | <b>122</b> |
| 21.1 Performance Tactics.....          | 122        |
| 21.2 Performance SLOs.....             | 123        |
| 21.3 Bundle Analysis.....              | 123        |
| <b>22. Accessibility.....</b>          | <b>126</b> |
| 22.1 Accessibility Rules.....          | 126        |
| 22.2 Automated Testing.....            | 128        |
| <b>23. Internationalization.....</b>   | <b>131</b> |
| 23.1 i18n Configuration.....           | 132        |
| 23.2 i18n Rules.....                   | 132        |
| 23.3 Translation File Example.....     | 133        |
| <b>24. Testing.....</b>                | <b>137</b> |
| 24.1 Test Pyramid.....                 | 137        |
| 24.2 Test Pyramid Diagram.....         | 138        |
| 24.3 Testing Rules.....                | 139        |

|                                    |            |
|------------------------------------|------------|
| 24.4 Component Test Example.....   | 141        |
| <b>25. Deployment.....</b>         | <b>145</b> |
| 25.1 Pre-Deployment Checklist..... | 146        |
| 25.2 Deployment Topology.....      | 146        |
| 25.3 CI/CD Pipeline.....           | 146        |
| <b>26. Coding Standards.....</b>   | <b>149</b> |
| 26.1 Standards Catalog.....        | 150        |
| 26.2 Pre-commit Hooks.....         | 152        |
| 26.3 PR Review Checklist.....      | 154        |
| <b>27. Folder Standards.....</b>   | <b>156</b> |
| 27.1 Folder Standards Catalog..... | 156        |
| 27.2 ESLint Import Boundaries..... | 158        |
| <b>28. Glossary.....</b>           | <b>162</b> |
| <b>29. Document Control.....</b>   | <b>162</b> |
| 29.1 Version History.....          | 162        |
| 29.2 Approval Matrix.....          | 162        |
| 29.3 Review Cadence.....           | 162        |
| 29.4 Distribution List.....        | 163        |
| 29.5 Companion Documents.....      | 164        |
| 29.6 Deliverables.....             | 165        |

*Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."*

# 1. Frontend Overview

## 1.1 Purpose

हा Document PreOne च्या Frontend System ची complete technical blueprint प्रदान करतो. Frontend हा PreOne platform चा user-facing layer आहे — teachers, parents, administrators, आणि platform operators सोबत direct interact करतो. या document चा primary goal म्हणजे frontend architecture, patterns, standards, आणि tooling ची एकच source of truth स्थापित करणे जी implementation, review, आणि onboarding साठी reference म्हणून वापरली जाऊ शकेल.

Frontend Architecture (FA) हा Vision, Master PRD, DDD, ADR, UI Design Philosophy, API Contract Catalog, आणि Backend Technical Design च्या वर build होतो. या document मधून पुढील production artifacts तयार होतील: Complete Next.js Project Structure, 14 Business Domain Modules, App Router Architecture, Global Layout System, State Management Strategy, API Client Layer, Authentication & Session Management, RBAC Guards (Route, Screen, Component, Button), Theme Engine & White Label Support, Form & Validation Framework, Enterprise Component Library, Dashboard & Data Table Standards, Performance Optimization Guide, Testing Strategy, आणि Frontend Coding Standards.

Frontend चा core philosophy म्हणजे type safety end-to-end (TypeScript + Zod + OpenAPI codegen), component-driven development (Storybook-first), accessibility-by-default (WCAG 2.2 AA), आणि progressive enhancement (server components + selective client hydration). ही philosophy enterprise-grade reliability + developer productivity दोन्ही सुनिश्चित करते.

## 1.2 Scope

हा document फक्त Web Portal (admin/teacher dashboard) च्या architecture cover करतो. खालील explicitly out of scope आहेत:

- Parent Mobile App (React Native) — separate mobile architecture document मध्ये.
- Marketing Website — separate Next.js app with different stack (MDX + content layer).
- Browser Extensions — out of current scope.
- Embedded Widgets (for third-party integration) — future scope.

## 1.3 Audience

हा document खालील audiences साठी लिखित आहे:

- **Frontend Engineers:** Primary reference — module structure, patterns, coding standards, testing strategy implementation साठी.
- **Tech Leads / Frontend Architects:** Architecture enforcement — folder boundaries, state management rules, performance budgets review साठी.
- **QA Engineers:** Testing pyramid understanding, E2E patterns, accessibility testing strategy साठी.
- **UI/UX Designers:** Component library structure, Storybook workflow, theme engine integration साठी.
- **DevOps / SRE:** Deployment readiness, performance monitoring, error tracking setup साठी.
- **AI Code Assistants:** Each section structured as a prompt-able block; AI tools च्या context मध्ये दिल्यावर accurate Next.js code generation होते.
- **New Hires:** Onboarding reference — first 2 weeks मध्ये वाचून frontend mental model तयार होतो.

## 1.4 Document Conventions

- Module names: kebab-case (student, attendance, identity).
- Component names: PascalCase (StudentCard, CreateStudentForm).
- File names: kebab-case (student-card.tsx, not StudentCard.tsx).
- Hook names: camelCase starting with 'use' (useStudents, usePermissions).
- Type names: PascalCase (StudentProfile, FeeBreakdown).
- Constant names: UPPER\_SNAKE\_CASE (MAX\_FILE\_SIZE).
- REST endpoints: kebab-case plural (/v1/students, /v1/attendance-records).
- Code blocks: monospace font (Consolas); tree diagrams use <sup>L</sup> – characters.
- Bilingual content: Marathi business context + English technical terms preserved exactly as user provided.

## 1.5 Related Documents

PreOne document series मध्ये FA चे specific position आहे. खालील documents FA चे predecessors आहेत:



| Document                 | Version | Status              | Relationship to FA                                                                         |
|--------------------------|---------|---------------------|--------------------------------------------------------------------------------------------|
| Vision Document          | v1.0    | Vision Freeze       | Strategic intent — FA ची non-functional requirements (accessibility, i18n) Vision शी align |
| Master PRD               | v1.0    | Product Freeze      | Functional requirements — FA चे screens + user flows PRD च्या FRs चे direct mapping        |
| DDD                      | v1.0    | Architecture Freeze | Domain model — FA चे 14 modules DDD च्या bounded contexts चे direct mirror                 |
| ADR Catalog              | v1.0    | Architecture Freeze | Decisions — FA प्रत्येक architectural choice ला ADR reference करतो                         |
| UI Design Philosophy     | v1.0    | Design Freeze       | Theme engine, component patterns, accessibility standards                                  |
| API Contract Catalog     | v1.0    | API Freeze          | Endpoint contracts — FA च्या API services चे spec                                          |
| Backend Technical Design | v1.0    | Backend Freeze      | Backend architecture — FA च्या auth + RBAC patterns शी aligned                             |

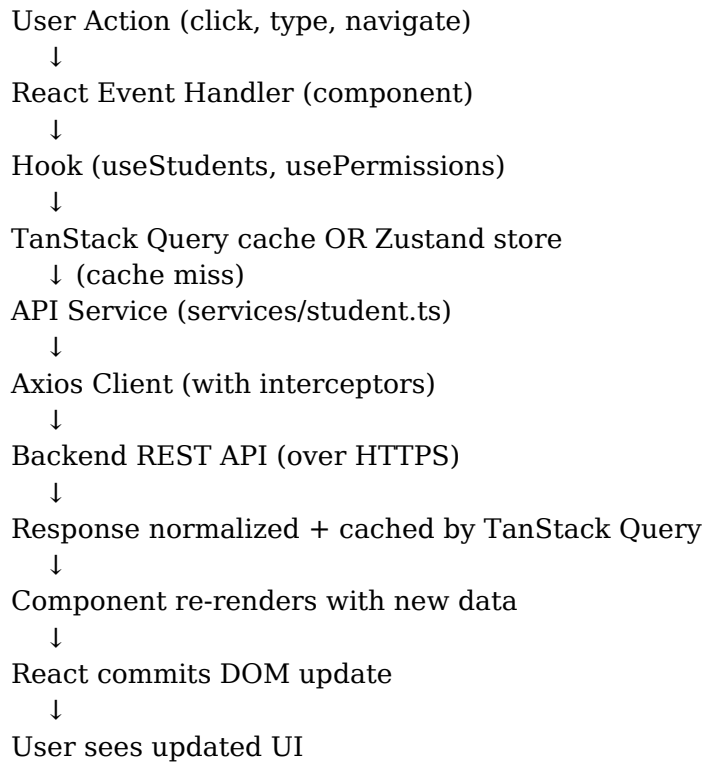
## 2. Technology Stack

PreOne frontend चा technology stack enterprise-grade reliability, developer productivity, आणि AI-assisted development optimize करण्यासाठी choose केला आहे. प्रत्येक choice ला ADR backing आहे जो trade-offs documented करतो. Stack चा core philosophy म्हणजे type safety end-to-end (TypeScript + Zod + OpenAPI codegen), component-driven development (ShadCN +

Storybook), आणि progressive enhancement (React Server Components + selective hydration).

## 2.1 Stack Pipeline

User interaction चा flow stack च्या प्रत्येक layer through जातो. ही pipeline understanding user experience आणि performance characteristics समजण्यासाठी critical आहे:



## 2.2 Component Catalog

खालील table सर्व stack components ची authoritative list आहे. नवीन component introduce करताना या table मध्ये entry जोडावी लागेल आणि corresponding ADR draft करावा लागेल:

| Component | Version | Role           | Highlights                                                               | Why Chosen                                                                |
|-----------|---------|----------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Next.js   | v16     | Meta-Framework | App Router, RSC, Server Actions, Edge Runtime, ISR, Partial Prerendering | Production-grade React framework; built-in optimizations; Vercel-grade DX |

| Component       | Version | Role                 | Highlights                                                                         | Why Chosen                                                       |
|-----------------|---------|----------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------|
| React           | v19     | UI Library           | Concurrent rendering, use(), Actions, useOptimistic, useFormStatus, useActionState | Latest stable; enables progressive enhancement + streaming       |
| TypeScript      | v5.5+   | Language             | Strict mode, path aliases, satisfies, const generics, decorators                   | Type safety end-to-end; backend types shared via OpenAPI codegen |
| Tailwind CSS    | v4      | Styling              | JIT, container queries, native cascade layers, oklch() colors, theme variables     | Utility-first velocity; design tokens via CSS vars               |
| ShadCN UI       | v2.x    | Component Foundation | Radix primitives, copy-into-repo, fully themable, accessible                       | Own the code; no vendor lock-in; AI-friendly composability       |
| TanStack Query  | v5      | Server State         | Suspense mode, infinite queries, mutations, optimistic updates, devtools           | Best-in-class data fetching + cache invalidation                 |
| Zustand         | v5      | Client State         | Slices, persist, devtools, middleware, selectors                                   | Minimal boilerplate; no provider hell; SSR-friendly              |
| React Hook Form | v7      | Form Management      | Resolver pattern, controlled/uncontrolled, field arrays, watch                     | Performance via uncontrolled refs; tiny re-render scope          |

| Component             | Version      | Role              | Highlights                                                        | Why Chosen                                                 |
|-----------------------|--------------|-------------------|-------------------------------------------------------------------|------------------------------------------------------------|
| Zod                   | v3           | Schema Validation | Infer, transform, refinements, discriminated unions, brand        | Single source of truth for client + server validation      |
| Axios                 | v1.7         | HTTP Client       | Interceptors, transformers, cancel tokens, retry, progress        | Mature interceptor ecosystem; per-request config           |
| Socket.IO Client      | v4           | Real-Time         | Rooms, namespaces, ack callbacks, reconnection, adapter           | Same protocol as backend; multi-instance via Redis adapter |
| NextAuth.js           | v5 (Auth.js) | Auth Layer        | JWT strategy, refresh rotation, custom providers, edge middleware | Standards-based; integrates with backend JWT flow          |
| next-intl             | v3           | i18n              | ICU MessageFormat, plurals, dates, numbers, locale routing        | App Router native; type-safe message keys                  |
| Recharts              | v2           | Charts            | Composable, responsive, custom tooltips, animation control        | React-native API; integrates with theme tokens             |
| TanStack Table        | v8           | Data Tables       | Headless, row models, column sizing, virtualization, sorting      | Headless = full styling control; SSR-ready                 |
| react-hook-form + zod | —            | Form Schema       | zodResolver, infer Input/Output types from schema                 | Schema drives UI + API contract                            |

| Component         | Version | Role             | Highlights                                                  | Why Chosen                                        |
|-------------------|---------|------------------|-------------------------------------------------------------|---------------------------------------------------|
| Vitest            | v2      | Unit Test Runner | Native ESM, jsdom, snapshot, coverage v8, in-source testing | Vite-powered fast startup; Jest API compatible    |
| Playwright        | v1.45+  | E2E + Visual     | Cross-browser, trace viewer, component testing, auto-wait   | Modern E2E; visual regression via screenshot diff |
| Storybook         | v8      | Component Dev    | CSF 3.0, addons, interaction tests, chromatic integration   | Living component documentation; design system hub |
| ESLint + Prettier | v9      | Lint/Format      | Flat config, import boundaries, react-hooks, jsx-a11y       | Architecture enforcement via import rules         |

## 2.3 Version Pinning Strategy

Production stability साठी सर्व dependencies pinned आहेत — caret ranges (~) फक्त patch updates allow करतात, minor/major bumps ला explicit review लागतो. Lockfile (bun.lock) committed आहे आणि Renovate Bot weekly PRs raise करतो. Next.js major upgrades (15 → 16 → 17) ला dedicated migration sprint allocate केला जातो — codemods + breaking changes review सह.

## 2.4 Type Sharing with Backend

Frontend आणि backend च्या वर type drift prevent करण्यासाठी OpenAPI codegen pipeline आहे. Backend च्या Swagger spec पासून frontend TypeScript types auto-generate होतात. हे types frontend च्या API services + Zod schemas चा foundation आहेत. Type drift CI मध्ये detect होतो — backend breaking change झाल्यास frontend build fail होतो:

```

Backend (NestJS + @nestjs/swagger)
  ↓ generates OpenAPI spec
openapi.json (committed to repo)
  ↓ codegen via openapi-typescript

```

```

types/api-contracts.ts (auto-generated, never hand-edited)
  ↓ imported by
services/*.ts (typed API clients)
  ↓ imported by
modules/*/hooks/*.ts (typed query hooks)
  ↓ consumed by
Components (typed props)

```

### 3. Project Structure

Project structure हे developer navigation चा foundation आहे. PreOne मध्ये strict folder convention आहे जो ESLint import boundaries ने enforce केला जातो — कोणतीही file wrong folder मध्ये place केल्यास lint error येतो आणि build fail होतो. ही discipline onboarding friction कमी करते आणि code discoverability वाढवते.

#### 3.1 Top-Level Layout

apps/web/ folder च्या खालील top-level directories आहेत. प्रत्येक directory ची specific responsibility आहे आणि cross-directory imports नियमित आहेत:

```

apps/web/
├── app/           # Next.js App Router
├── modules/       # Business domain modules
├── components/    # Shared UI components
├── layouts/       # Layout shells
├── hooks/         # Shared hooks
├── lib/           # Framework glue (singletons, config)
├── services/      # API service layer
├── store/         # Zustand stores (UI state)
├── providers/     # Context providers
├── styles/        # Global CSS + theme tokens
├── types/         # Shared TypeScript types
├── utils/         # Pure utility functions
├── assets/        # Static assets (images, fonts, icons)
└── middleware.ts  # Next.js edge middleware

```

#### 3.2 Top-Level Directory Roles

| Directory | Responsibility                                                        | Notes                    |
|-----------|-----------------------------------------------------------------------|--------------------------|
| app/      | Next.js App Router — routes, layouts, loading, error, page components | Routing root; file-based |

| Directory     | Responsibility                                                    | Notes                       |
|---------------|-------------------------------------------------------------------|-----------------------------|
| modules/      | Business domain modules (one folder per bounded context)          | Feature-first organization  |
| components/   | Shared UI components — base primitives + composite                | Cross-module reusable       |
| layouts/      | Layout shells — AuthLayout, DashboardLayout, PrintLayout          | Page-level wrappers         |
| hooks/        | Shared hooks — useDebounce, usePagination, usePermissions         | Cross-module reusable       |
| lib/          | Framework glue — queryClient, axiosClient, authConfig, i18nConfig | Singletons + config         |
| services/     | API service layer — per-domain API clients                        | REST + WebSocket clients    |
| store/        | Zustand stores — global UI state slices                           | Client state                |
| providers/    | Context providers — QueryProvider, ThemeProvider, AuthProvider    | App-wide context wiring     |
| styles/       | Global CSS, Tailwind layers, theme tokens, animations             | Design system foundation    |
| types/        | Shared TypeScript types — DTOs, enums, brand types                | Cross-module type contracts |
| utils/        | Pure utility functions — formatters, parsers, validators          | Side-effect-free helpers    |
| assets/       | Static assets — images, fonts, icons, illustrations               | Bundled at build            |
| middleware.ts | Next.js edge middleware — auth gate, locale redirect              | Runs before every route     |

### 3.3 Monorepo Layout

PreOne हा monorepo आहे — apps/ मध्ये multiple applications (web, mobile-future, marketing) आणि packages/ मध्ये shared packages (ui, utils, types, config). हे setup Turborepo ने manage होतो — task orchestration, caching, आणि parallel execution सह. प्रत्येक app व app-level dependencies own करतो, shared packages ची versioning independent आहे:

```

preone/
├── apps/
│   ├── web/           # Next.js admin portal
│   ├── marketing/     # Public marketing site
│   └── mobile/        # Future React Native app
├── packages/
│   ├── ui/            # Shared component library
│   ├── types/         # Shared TypeScript types
│   ├── utils/         # Shared utilities
│   ├── config/        # Shared config (eslint, tsconfig, tailwind)
│   └── api-client/    # Generated API client
├── turbo.json
└── package.json

```

## 4. App Router

Next.js 16 चा App Router हा PreOne चा primary routing system आहे. App Router file-based routing uses करतो — filesystem structure च mirror URL structure. Server Components by default, selective 'use client' directive for interactive components. हे approach performance optimal करते — minimum JS shipped to client, maximum work on server.

### 4.1 Route Groups

App Router मध्ये route groups (parenthesized folders) URL structure ला affect न करते logical grouping enable करतात. PreOne मध्ये दोन primary route groups आहेत — (auth) for unauthenticated pages (login, forgot password) आणि (dashboard) for authenticated pages (students, finance, etc.). प्रत्येक group चा आपला layout आहे:

```

app/
├── (auth)/
│   ├── layout.tsx      # AuthLayout (minimal chrome)
│   ├── login/page.tsx
│   ├── forgot-password/page.tsx
│   └── reset-password/page.tsx
├── (dashboard)/
│   ├── layout.tsx      # DashboardLayout (sidebar + header)
│   ├── dashboard/page.tsx
│   ├── student/
│   │   └── page.tsx    # Student list

```



```

|   |   | [id]/page.tsx      # Student detail
|   |   | — admissions/
|   |   | — finance/
|   |   | — inventory/
|   |   | — attendance/
|   |   | — academics/
|   |   | — communication/
|   |   | — hr/
|   |   | — administration/
|   |   | — reports/
|   |   | — settings/
|   |   | — platform/
|   | — (print)/
|   |   | — reports/[id]/page.tsx # Print-optimized layout
| — layout.tsx                    # RootLayout (HTML shell)
| — loading.tsx                   # Global loading fallback
| — error.tsx                     # Global error boundary
| — global-error.tsx              # Root-level error
| — not-found.tsx                 # 404 page

```

## 4.2 App Router Features

PreOne मध्ये App Router च्या खालील features use होतात. हे features ADR-FA-003 मध्ये documented आहेत:

- **Server Components (Default):** Components render on server, zero JS shipped. Most pages RSC-only — data fetched server-side, HTML streamed.
- **Route Groups:** Logical grouping without URL impact — (auth), (dashboard), (print).
- **Nested Layouts:** Layouts nest — RootLayout > DashboardLayout > StudentDetailLayout. Persistent UI across route changes.
- **Dynamic Routes:** [id] segment for entity detail pages. Generate static params for known IDs (ISR).
- **Parallel Routes:** @analytics, @feed slots rendered side-by-side. Used in dashboard for independent data loading.
- **Intercepting Routes:** (..)modal intercepts navigation — e.g., photo modal opens over gallery. Used in student profile.
- **Loading States:** loading.tsx auto-wraps route in Suspense. Skeletons during data fetch.
- **Error Boundaries:** error.tsx catches errors within route subtree. Recovery actions + error ID.

- **Server Actions:** Form submissions via Server Actions — no API endpoint needed.  
Used for simple mutations.
- **Streaming:** React Suspense + Server Components — progressive HTML streaming.  
Faster TTFB.

### 4.3 Route Metadata

प्रत्येक route ची metadata declare केलेली असते — title, description, permissions, sidebar config. ही metadata generateMetadata() function through populate होते. middleware.ts या metadata ला read करून permission check perform करतो:

```
// app/(dashboard)/student/[id]/page.tsx
export const metadata = {
  title: 'Student Profile',
  description: 'View + edit student details',
  permissions: ['STUDENT_READ'], // custom field
  sidebar: { key: 'students', label: 'Students', icon: 'users' },
};

export default async function StudentProfilePage({
  params,
}: {
  params: { id: string };
}) {
  const student = await fetchStudent(params.id); // server-side fetch
  return <StudentProfile student={student} />;
}
```

## 5. Module Structure

PreOne चा प्रत्येक business domain identical folder template follow करतो. हे consistency onboarding friction कमी करते — developer एक module शिकल्यावर दुसरा module navigate करणे intuitive होते. प्रत्येक Business Domain स्वतंत्र module असेल. Module ची structure DDD च्या bounded context शी 1:1 align आहे — 14 modules = 14 bounded contexts.

### 5.1 Module Template (student/ example)

खालील tree student/ module चे complete structure दाखवते. हेच pattern सर्व 14 modules साठी apply होते — फक्त folder name बदलते:

```
modules/student/
├── api/           # Module-specific API client functions
├── components/    # Module UI components
├── pages/         # Page compositions
└── forms/        # Form components (RHF + Zod)
```

|                |                                          |
|----------------|------------------------------------------|
| └─ hooks/      | # Module hooks (TanStack Query wrappers) |
| └─ services/   | # Business logic (pure functions)        |
| └─ store/      | # Module-local Zustand slice (UI state)  |
| └─ types/      | # Module DTOs + types                    |
| └─ validation/ | # Zod schemas (source of truth)          |
| └─ utils/      | # Module-scoped helpers                  |
| └─ index.ts    | # Public API barrel                      |

## 5.2 Directory Responsibilities

प्रत्येक sub-directory ची specific responsibility आहे. खालील table सर्व directories चे purpose + boundaries document करते:

| Directory   | Responsibility                                                       | Example                                            |
|-------------|----------------------------------------------------------------------|----------------------------------------------------|
| api/        | Module-specific API client functions                                 | e.g., fetchStudents, createStudent, searchStudents |
| components/ | Module UI components — StudentCard, StudentList, StudentDetail       | Module-scoped, not shared                          |
| pages/      | Page-level compositions (used inside App Router routes)              | Compose module components + hooks                  |
| forms/      | Form components — CreateStudentForm, EditStudentForm, BulkImportForm | React Hook Form + Zod                              |
| hooks/      | Module hooks — useStudents, useStudentMutations, useStudentFilters   | Encapsulate TanStack Query                         |
| services/   | Module services — business logic that doesn't fit components         | Pure functions + side effects                      |
| store/      | Module-local Zustand slice (UI state only — no server data)          | e.g., student list filters                         |
| types/      | Module-specific types — StudentProfile, StudentListFilters           | Module DTOs                                        |

| Directory   | Responsibility                                                                | Example                |
|-------------|-------------------------------------------------------------------------------|------------------------|
| validation/ | Zod schemas —<br>CreateStudentSchema,<br>UpdateStudentSchema,<br>FilterSchema | Single source of truth |
| utils/      | Module utility functions —<br>formatStudentName,<br>computeAge                | Module-scoped helpers  |
| index.ts    | Public API barrel — exports<br>only what other modules<br>may import          | Module encapsulation   |

### 5.3 Module List

PreOne मध्ये 14 business domain modules आहेत — DDD च्या 14 bounded contexts शी 1:1 mapping. प्रत्येक module चे independent ownership, release cadence, आणि test suite आहे:

| # | Module        | Primary Domain                                   | Approximate Pages |
|---|---------------|--------------------------------------------------|-------------------|
| 1 | identity      | Login, Profile,<br>Settings, Tenant<br>Switch    | ~12               |
| 2 | crm           | Leads, Campaigns,<br>Conversions                 | ~15               |
| 3 | admissions    | Applications,<br>Counselling,<br>Approvals       | ~18               |
| 4 | student       | Student Lifecycle,<br>Profiles, Guardians        | ~22               |
| 5 | academics     | Curriculum,<br>Observations, Report<br>Cards     | ~20               |
| 6 | attendance    | Daily Attendance,<br>Arrival, Pickup,<br>Reports | ~12               |
| 7 | communication | Announcements,<br>Chat, Broadcasts               | ~15               |

| #  | Module         | Primary Domain                                      | Approximate Pages |
|----|----------------|-----------------------------------------------------|-------------------|
| 8  | finance        | Fees, Invoices, Payments, Ledger, Reports           | ~28               |
| 9  | inventory      | Items, Stock, PR/PO/GRN, Vendors                    | ~18               |
| 10 | hr             | Staff, Payroll, Leave, Attendance, Reviews          | ~22               |
| 11 | administration | Assets, Maintenance, Visitors, Transport            | ~15               |
| 12 | reports        | Cross-domain Reports, Analytics, KPIs               | ~15               |
| 13 | settings       | Academic Years, Calendars, Configs, Feature Flags   | ~10               |
| 14 | platform       | Subscriptions, Billing, Feature Flags, Integrations | ~12               |

## 5.4 Module Barrel Pattern

प्रत्येक module च्या index.ts file चा role म्हणजे module च्या public API define करणे. फक्त index.ts मधून exported symbols च दुसरा module import करू शकतो. यामुळे module encapsulation enforce होते — internal files refactor करणे safe होते. ESLint rule या discipline ला enforce करतो:

```
// modules/student/index.ts
export { StudentCard } from './components/student-card';
export { StudentList } from './components/student-list';
export { CreateStudentForm } from './forms/create-student-form';
export { useStudents, useStudent, useCreateStudent } from './hooks';
export { fetchStudents, createStudent } from './api';
export type { Student, StudentFilters, CreateStudentInput } from './types';

// Other modules import ONLY from this barrel:
// import { StudentCard, useStudents } from '@modules/student';
// NEVER from internal files:
// import { StudentCard } from '@modules/student/components/student-card'; // FORBIDDEN
```

## 6. Layout System

Layout system हे UI चा structural foundation आहे. PreOne मध्ये 8 distinct layouts आहेत — प्रत्येक layout ची specific responsibility आणि use case आहे. Layouts NestJS च्या nested layout pattern through compose होतात — RootLayout wraps AuthLayout wraps LoginPage.

### 6.1 Layout Catalog

| Layout          | File                   | Purpose                                        | Notes                    |
|-----------------|------------------------|------------------------------------------------|--------------------------|
| RootLayout      | app/layout.tsx         | HTML shell, font loading, providers            | Always wraps everything  |
| AuthLayout      | (auth)/layout.tsx      | Login, Forgot Password, Reset — minimal chrome | No sidebar, no header    |
| DashboardLayout | (dashboard)/layout.tsx | Sidebar + Header + Breadcrumb + Content        | Most pages use this      |
| PrintLayout     | (print)/layout.tsx     | Print-optimized — no nav, A4 size              | Report cards, invoices   |
| ModalLayout     | —                      | Dialog/Drawer rendered in portal               | Forced-focus overlays    |
| ErrorLayout     | error.tsx              | Full-page error UI with recovery actions       | Catches route errors     |
| LoadingLayout   | loading.tsx            | Skeleton/spinner during route transitions      | Suspense fallback        |
| NotFoundLayout  | not-found.tsx          | 404 page with helpful navigation               | Catches unmatched routes |

### 6.2 Layout Slots

DashboardLayout हा सर्वात complex layout आहे — त्यात खालील slots आहेत. हे slots compose होऊन complete dashboard UI तयार होते. प्रत्येक slot independent component आहे ज्याला individually test आणि replace करता येते:

| Slot        | Purpose                                                                    | Used In            |
|-------------|----------------------------------------------------------------------------|--------------------|
| Header      | Top bar — logo, school switcher, branch switcher, notifications, user menu | DashboardLayout    |
| Sidebar     | Primary navigation — RBAC-driven menu tree, collapsible, multi-level       | DashboardLayout    |
| Breadcrumb  | Current location trail — Home / Students / Profile                         | DashboardLayout    |
| Content     | Main page content — varies per route                                       | All layouts        |
| Footer      | Minimal — version, copyright, support link                                 | DashboardLayout    |
| PageHeader  | Page title, description, primary actions                                   | Content (top)      |
| PageActions | Contextual actions — Export, Create, Filter                                | PageHeader (right) |

### 6.3 Layout Composition

Layouts chain तयार करतात — प्रत्येक outer layout त्याच्या inner content ला wrap करतो. ही composition React Context through propagate होते — inner components outer layout चा context access करू शकतात (e.g., sidebar collapse state):

```

<RootLayout>                // app/layout.tsx
  <html>
    <body>
      <Providers>              // QueryProvider, ThemeProvider, AuthProvider
        <NextIntlClientProvider>
          {children}           // (auth) or (dashboard) layout
        </NextIntlClientProvider>
      </Providers>
    </body>
  </html>
</RootLayout>

<DashboardLayout>            // (dashboard)/layout.tsx
  <Header />                  // school/branch/year switchers, user menu
  <Sidebar />                  // RBAC-filtered menu
  <Breadcrumb />               // auto-generated
  <main>{children}</main>      // actual page content
  <Footer />

```

</DashboardLayout>

## 7. State Management

State management हे frontend architecture चा core आहे. PreOne मध्ये state चे three categories आहेत — Server State (TanStack Query), Client/UI State (Zustand), आणि Form State (React Hook Form). यांचे clear separation critical आहे — गल्लत केल्यास bugs आणि performance issues येतात.

### 7.1 Global State (Zustand)

Zustand मध्ये global UI state stores आहेत — user, permissions, theme, sidebar, notifications, current school/branch/year. हे state cross-component shared करायला लागते, पण server data नाही. Server data नेहमी TanStack Query मध्ये राहते — Zustand मध्ये कधीच. हे rule strict आहे:

| State                 | Type                                        | Description                                                          | Source                     |
|-----------------------|---------------------------------------------|----------------------------------------------------------------------|----------------------------|
| User                  | AuthUser   null                             | Current authenticated user — id, name, email, avatar                 | AuthProvider               |
| Permissions           | string[]                                    | Resolved permission codes — e.g., ['STUDENT_READ', 'INVOICE_CREATE'] | Loaded on login + cache    |
| Theme                 | 'light'   'dark'   'auto'                   | Active theme preference                                              | ThemeProvider + persisted  |
| Sidebar               | { collapsed: boolean, mobileOpen: boolean } | Sidebar collapse state                                               | UI-only, no persistence    |
| Notifications         | Notification[]                              | In-app notification stack (max 5 visible)                            | Auto-dismiss after 5s      |
| Current School        | School   null                               | Active school context (multi-school admin)                           | Switchable, persisted      |
| Current Branch        | Branch   null                               | Active branch context within school                                  | Switchable, persisted      |
| Current Academic Year | AcademicYear   null                         | Active academic year for filtering                                   | Switchable per school year |



## 7.2 Server State (TanStack Query)

TanStack Query सर्व server data handle करतो — students list, admissions pipeline, attendance grid, finance ledger. प्रत्येक query ची specific stale time, refetch strategy, आणि invalidation rules आहेत. Server state कधीच Zustand मध्ये mirror होत नाही — ते फक्त TanStack Query cache मध्ये राहते:

| Domain        | Module                     | Cache Strategy                          | Invalidation                     |
|---------------|----------------------------|-----------------------------------------|----------------------------------|
| Students      | identity + student modules | Student list, detail, search            | 5 min stale; refetch on focus    |
| Admissions    | admissions module          | Application pipeline, status, documents | 30s polling on active tab        |
| Attendance    | attendance module          | Daily attendance grid, monthly summary  | Live updates via Socket.IO       |
| Finance       | finance module             | Invoices, payments, ledger entries      | Invalidate on mutation           |
| Reports       | reports module             | Generated reports, KPIs, dashboards     | 60s stale; manual refresh button |
| Inventory     | inventory module           | Stock levels, POs, low-stock alerts     | 30s stale; refetch on focus      |
| HR            | hr module                  | Staff list, leave calendar, payroll     | Invalidate on mutation           |
| Communication | communication module       | Announcements, chat messages            | Socket.IO live updates           |

## 7.3 State Management Rules

खालील rules state management discipline सुनिश्चित करतात. यातील बऱ्याच rules ESLint ने enforce होतात — बाकी PR review मध्ये check होतात:

- Server data → TanStack Query (NEVER in Zustand)
- UI state (toggles, filters, modals) → Zustand (sliced, persisted selectively)
- Form state → React Hook Form (uncontrolled refs, not React state)
- URL state (page, sort, filter) → useSearchParams + Next.js router
- Local component state → useState/useReducer (ephemeral, not shared)
- Cross-component shared UI → Zustand slice (e.g., sidebar collapse)

- Server-derived state → useMemo (never re-fetched, derived from cache)
- Persistent preferences → Zustand persist middleware (theme, last school)
- Realtime updates → invalidate query on Socket.IO event
- Optimistic updates → TanStack Query onMutate + rollback onError

## 7.4 Zustand Store Example

Zustand store ची structure खालीलप्रमाणे आहे. एकाच store मध्ये multiple slices combine होतात — यामुळे single source of truth सुनिश्चित होते. Persist middleware user preferences (theme, last school) ला localStorage मध्ये save करतो:

```
interface AppStore {
  // State
  user: AuthUser | null;
  permissions: string[];
  theme: 'light' | 'dark' | 'auto';
  sidebar: { collapsed: boolean; mobileOpen: boolean };
  currentSchool: School | null;
  currentBranch: Branch | null;
  currentAcademicYear: AcademicYear | null;
  notifications: Notification[];

  // Actions
  setUser: (user: AuthUser | null) => void;
  setPermissions: (perms: string[]) => void;
  setTheme: (theme: 'light' | 'dark' | 'auto') => void;
  toggleSidebar: () => void;
  switchSchool: (school: School) => void;
  switchBranch: (branch: Branch) => void;
  addNotification: (notif: Notification) => void;
  dismissNotification: (id: string) => void;
}

export const useAppStore = create<AppStore>()(
  persist(
    (set) => ({
      // initial state + actions
    }),
    { name: 'preone-app', partialize: (s) => ({ theme: s.theme, currentSchool: s.currentSchool }) }
  )
);
```

## 8. API Layer

API Layer हा frontend आणि backend च्या मध्ये communication चा structured bridge आहे. Page कधीच directly axios ला call करत नाही — ते फक्त hooks शी interact करते. Hooks API services शी, services axios client शी, axios client backend REST API शी. ही layered architecture testability + maintainability सुनिश्चित करते.

### 8.1 API Layer Flow

Request चा flow खालीलप्रमाणे आहे. प्रत्येक layer ची specific responsibility आहे — गल्लत केल्यास debugging कठीण होते:

```

Page Component (renders)
  ↓
useStudents() hook (TanStack Query wrapper)
  ↓
studentApi.list(filters) function call
  ↓
axiosClient.get('/v1/students', { params: filters })
  ↓ (request interceptor: attach JWT + traceId + tenantId)
Backend REST API (NestJS)
  ↓
Response (success or error)
  ↓ (response interceptor: normalize errors)
TanStack Query cache populated (or error thrown)
  ↓
Component re-renders with data (or shows error state)

```

### 8.2 API Service Catalog

services/ directory मध्ये per-domain API client files आहेत. प्रत्येक file त्या domain चे सर्व endpoints encapsulate करतो. नवीन endpoint add करताना corresponding service file मध्ये entry जोडावी लागेल:

| File       | Purpose                                                        | Notes                |
|------------|----------------------------------------------------------------|----------------------|
| api.ts     | Core axios instance — interceptors, base URL, error normalizer | Single shared client |
| auth.ts    | Login, refresh, logout, me, permission resolve                 | Auth namespace       |
| student.ts | CRUD + search + bulk + export + import                         | Student module       |

| File             | Purpose                                                    | Notes                |
|------------------|------------------------------------------------------------|----------------------|
| finance.ts       | Invoices, payments, receipts, refunds, ledger, GST returns | Finance module       |
| attendance.ts    | Mark, edit, monthly summary, bulk mark                     | Attendance module    |
| communication.ts | Announcements, chat, broadcast, templates                  | Communication module |
| admissions.ts    | Applications, counselling, approval, documents             | Admissions module    |
| academics.ts     | Curriculum, observations, report cards, PTM                | Academics module     |
| inventory.ts     | Items, stock, PR/PO/GRN, vendors                           | Inventory module     |
| hr.ts            | Staff, leave, payroll, attendance                          | HR module            |
| reports.ts       | Report generation, download, schedule                      | Reports module       |
| platform.ts      | Subscriptions, billing, feature flags, integrations        | Platform module      |

### 8.3 Axios Interceptor Rules

Axios interceptors हे cross-cutting concerns handle करतात — auth, tracing, error normalization. खालील rules ADR-FA-008 मध्ये documented आहेत:

- Request interceptor: attach Authorization header from auth store
- Request interceptor: attach traceId header (UUID v7) for distributed tracing
- Request interceptor: attach tenantId + branchId from context store
- Response interceptor (2xx): pass through
- Response interceptor (401): attempt silent refresh, retry original request
- Response interceptor (403): redirect to /forbidden
- Response interceptor (429): backoff retry with jitter (3 attempts max)
- Response interceptor (5xx): throw standardized ApiError
- Response interceptor (network): throw NetworkError with offline hint
- All errors normalized to ApiError shape for component consumption

## 8.4 Service Example

API service file चा structure खालीलप्रमाणे आहे. प्रत्येक function typed input accept करतो आणि typed output return करतो — types OpenAPI codegen पासून येतात. यामुळे frontend आणि backend च्या वर type drift prevent होते:

```
// services/student.ts
import { axiosClient } from '@lib/axios-client';
import type { Student, CreateStudentInput, StudentFilters } from
'@/types/api-contracts';

export const studentApi = {
  list: (filters: StudentFilters) =>
    axiosClient.get<Page<Student>>('/v1/students', { params: filters }),

  getById: (id: string) =>
    axiosClient.get<Student>(`/v1/students/${id}`),

  create: (input: CreateStudentInput) =>
    axiosClient.post<Student>('/v1/students', input),

  update: (id: string, input: Partial<CreateStudentInput>) =>
    axiosClient.patch<Student>(`/v1/students/${id}`, input),

  delete: (id: string) =>
    axiosClient.delete<void>(`/v1/students/${id}`),

  search: (query: string) =>
    axiosClient.get<Page<Student>>('/v1/students/search', { params: { q:
query } })),

  bulkCreate: (inputs: CreateStudentInput[]) =>
    axiosClient.post<BatchResult<Student>>('/v1/students/bulk', { items:
inputs })),

  export: (filters: StudentFilters, format: 'xlsx' | 'csv' | 'pdf') =>
    axiosClient.post<{ jobId: string }>('/v1/students/export', { filters, format }),
};
```

## 9. Authentication

Authentication हे security चा foundation आहे. PreOne JWT-based authentication use करतो — short-lived access tokens + long-lived refresh tokens. Access tokens memory मध्ये राहतात (XSS-resistant), refresh tokens HttpOnly cookies मध्ये (CSRF-resistant). Silent refresh user ला invisible राहतो — session continuous असतो.

## 9.1 Authentication Flow

Login चा complete flow खालीलप्रमाणे आहे. प्रत्येक step specific responsibility handle करते — error कोणत्याही step पर येऊ शकतो आणि user-friendly message शी translate होतो:

```

User enters email + password on /login
↓
POST /v1/auth/login
↓
Server returns: {
  accessToken (15min, RS256 signed),
  refreshToken (30d, HttpOnly cookie, rotated),
  user: { id, name, email, avatar }
}
↓
Store accessToken in memory (Zustand) — NOT localStorage
↓
Resolve permissions: GET /v1/auth/permissions
↓
Cache permissions in Zustand + sessionStorage
↓
Generate menu tree based on permissions
↓
Redirect to /dashboard

```

## 9.2 Authentication Features

PreOne च्या authentication system मध्ये खालील features आहेत. हे features enterprise-grade UX आणि security दोन्ही सुनिश्चित करतात:

| Feature                     | Mechanism                                                                    | Benefit                           |
|-----------------------------|------------------------------------------------------------------------------|-----------------------------------|
| Silent Refresh              | Axios interceptor catches 401, calls /auth/refresh, retries original request | User never sees login mid-session |
| Auto Logout                 | On 401 after refresh attempt fails — clear state, redirect to /login         | Security: prevent stale token use |
| Session Recovery            | On page reload — validate accessToken, re-resolve permissions if expired     | Seamless UX across reloads        |
| Multi-school Context Switch | School switcher in header; switches tenant context + clears server cache     | Admin users with multiple schools |

| Feature                  | Mechanism                                                                         | Benefit                        |
|--------------------------|-----------------------------------------------------------------------------------|--------------------------------|
| Branch Switch            | Branch switcher in header; updates context.branchId; invalidates affected queries | Multi-branch schools           |
| Academic Year Switch     | Year switcher in header; updates context.academicYearId                           | Filter scope for academic data |
| Idle Timeout             | 30 min idle → soft warning at 25 min → logout at 30 min                           | DPDP Act compliance            |
| Concurrent Session Limit | Max 3 concurrent sessions per user; oldest auto-revoked                           | Prevent session sharing        |

### 9.3 Token Storage Strategy

Token storage हे security चा critical aspect आहे. खालील table दाखवते कोणता token कोठे store होतो आणि का. या strategy मुळे XSS आणि CSRF दोन्ही attacks पासून protection मिळते:

| Token            | Storage                            | Why                                                                          | Lifetime         |
|------------------|------------------------------------|------------------------------------------------------------------------------|------------------|
| Access Token     | Memory (Zustand store)             | XSS-resistant (no localStorage); lost on refresh but silent refresh recovers | 15 min           |
| Refresh Token    | HttpOnly Cookie                    | JS cannot read (XSS-resistant); SameSite=Strict (CSRF-resistant)             | 30 days, rotated |
| Permissions      | Zustand + sessionStorage           | Fast access on page reload; cleared on logout                                | Session          |
| User Profile     | Zustand + sessionStorage           | Fast access; cleared on logout                                               | Session          |
| Theme Preference | localStorage (via Zustand persist) | Survives logout — applies to login page too                                  | Persistent       |

## 9.4 Multi-School Context Switch

Platform admin users एकाच पेक्षा जास्त schools manage करतात. School switcher header मध्ये आहे — switch करताना current school, branch, academic year context update होतो आणि सर्व server cache invalidate होतो. हे behavior ADR-FA-009 मध्ये documented आहे. Switch केल्यावर पहिला page dashboard असतो — mixed-context data confusion prevent करण्यासाठी:

```

User clicks school switcher in header
↓
Dropdown shows list of schools user has access to
↓ (user selects)
Update Zustand: currentSchool, currentBranch=null,
currentAcademicYear=null
↓
Clear TanStack Query cache (queryClient.clear())
↓
Re-fetch permissions for new school (if differs)
↓
Re-build sidebar menu based on new permissions
↓
Redirect to /dashboard (clean slate)

```

## 10. RBAC

Role-Based Access Control (RBAC) हे PreOne च्या security चा core mechanism आहे. Frontend मध्ये RBAC चे 7 layers आहेत — defense in depth. प्रत्येक layer different granularity पर enforce होतो — route (coarsest) पासून field (finest) पर्यंत. Screen, menu, button, आणि field-level authorization API Contract आणि ADR मध्ये define केलेल्या permission model शी aligned राहिल.

### 10.1 RBAC Layers

खालील table सर्व 7 layers ची catalog करते. प्रत्येक layer ची specific responsibility आहे — एक layer fail झाला तरी दुसरा defense देतो:

| Layer       | Mechanism                              | Purpose                                                      | When                         |
|-------------|----------------------------------------|--------------------------------------------------------------|------------------------------|
| Route Guard | middleware.ts + (dashboard)/layout.tsx | Block access to unauthorized routes — redirect to /forbidden | Before page render           |
| Menu Guard  | Sidebar menu builder                   | Hide menu items user has no permission for                   | On login + permission change |



| Layer           | Mechanism                                                   | Purpose                                       | When          |
|-----------------|-------------------------------------------------------------|-----------------------------------------------|---------------|
| Screen Guard    | Page-level <Can> wrapper or usePermission hook              | Show 403 fallback if user lacks screen access | Per page      |
| Component Guard | <Can permission="STUDENT_EDIT"> wrapper                     | Conditionally render component                | Per component |
| Button Guard    | <Can permission="INVOICE_DELETE"> wrapper                   | Hide/disable button                           | Per button    |
| Field Guard     | <Can permission="STUDENT_VIEW_AADHAAR"> wrapper             | Mask or hide sensitive fields                 | Per field     |
| API Guard       | Backend enforces (defense in depth) — frontend never trusts | Returns 403 if unauthorized                   | Per request   |

## 10.2 RBAC Examples

खालील examples दाखवतात कसे <Can> component वेगवेगळ्या scenarios मध्ये use होते. प्रत्येक example ची specific use case आहे — copy-paste ready:

| Code                                                               | Component        | Purpose                                                 |
|--------------------------------------------------------------------|------------------|---------------------------------------------------------|
| <Can permission="STUDENT_EDIT">                                    | <EditButton />   | Show edit button only if user can edit students         |
| <Can permission="FINANCE_EXPORT">                                  | <ExportButton /> | Show export button only if user can export finance data |
| <Can permission="STUDENT_VIEW_AADHAAR" fallback={<MaskedField />}> | <AadhaarField /> | Mask Aadhaar for users without explicit permission      |
| <Can role="ADMIN" anyOf=[["FINANCE_ADMIN", "PLATFORM_ADMIN"]]>     | <DeleteButton /> | Show delete button for admin roles                      |

| Code                            | Component                                         | Purpose                      |
|---------------------------------|---------------------------------------------------|------------------------------|
| usePermission('STUDENT_CREATE') | const canCreate = usePermission('STUDENT_CREATE') | Hook for programmatic checks |
| menuBuilder(permissions)        | Generate sidebar menu tree from permission list   | Filter menu items at runtime |

## 10.3 RBAC Rules

- Every screen must declare required permission in route metadata
- Every data-modifying button must be wrapped in <Can>
- Every sensitive field (Aadhaar, PAN, salary) must be field-guarded
- Menu items dynamically filtered — never hardcode visibility
- API responses may include permissions for context-aware UI
- Permission changes (role update) trigger menu re-build + cache invalidation
- Frontend RBAC is UX optimization — backend is source of truth
- Permission codes match backend ADR exactly (e.g., STUDENT\_EDIT, not student.edit)
- Route guard uses middleware (edge runtime) for instant redirect — no flash of content
- All <Can> components memoized to prevent re-render storms

## 10.4 <Can> Component Implementation

<Can> component चा implementation खालीलप्रमाणे आहे. हा component memoized आहे — permissions change झाल्याशिवाय re-render होत नाही. Fallback prop allow करते की user ला permission नसेल तर काय दाखवायचे (default: null):

```
interface CanProps {
  permission?: string | string[];
  role?: string | string[];
  anyOf?: string[]; // OR logic
  allOf?: string[]; // AND logic
  fallback?: React.ReactNode;
  children: React.ReactNode;
}

export const Can = memo(function Can({
  permission,
```

```

    role,
    anyOf,
    allOf,
    fallback = null,
    children,
  }: CanProps) {
    const permissions = useAppStore((s) => s.permissions);
    const userRole = useAppStore((s) => s.user?.role);

    const allowed = useMemo(() => {
      if (permission) {
        return Array.isArray(permission)
          ? permission.some((p) => permissions.includes(p))
          : permissions.includes(permission);
      }
      if (role) {
        return Array.isArray(role) ? role.includes(userRole) : role === userRole;
      }
      if (anyOf) return anyOf.some((p) => permissions.includes(p));
      if (allOf) return allOf.every((p) => permissions.includes(p));
      return false;
    }, [permission, role, anyOf, allOf, permissions, userRole]);

    return <>{allowed ? children : fallback}</>;
  });

```

## 11. Navigation

Navigation हे user experience चा critical part आहे. PreOne मध्ये multiple navigation mechanisms आहेत — primary sidebar, top header, breadcrumbs, command palette. प्रत्येक mechanism ची specific use case आहे आणि ते all together cohesive navigation experience तयार करतात.

### 11.1 Navigation Types

| Type            | Description                                            | Use Case            |
|-----------------|--------------------------------------------------------|---------------------|
| Primary Sidebar | Multi-level collapsible menu, RBAC-filtered            | All dashboard pages |
| Top Header      | School/Branch/Year switchers, notifications, user menu | Dashboard layout    |
| Breadcrumbs     | Auto-generated from route + page metadata              | Below header        |

| Type              | Description                                                                | Use Case        |
|-------------------|----------------------------------------------------------------------------|-----------------|
| In-page Tabs      | Tab navigation within a page (e.g., Student → Profile   Attendance   Fees) | Detail pages    |
| Pagination        | Cursor or page-based navigation for lists                                  | Data tables     |
| Command Palette   | Cmd+K fuzzy search across pages/actions                                    | Power users     |
| Quick Actions     | Floating action button for primary action on list pages                    | Mobile-friendly |
| Mobile Bottom Nav | Bottom tab bar on mobile breakpoints                                       | Responsive      |

## 11.2 Navigation Rules

- Sidebar collapses to icons-only on medium screens; hamburger on mobile
- Active route highlighted in sidebar + breadcrumb
- Breadcrumbs reflect actual navigation path, not URL structure
- External links open in new tab with rel="noopener noreferrer"
- Back button preserved via browser history; never broken by client-side routing
- Route transitions show loading.tsx skeleton immediately (no white flash)
- Navigation to unauthorized route shows 403 page (not silent redirect)
- Cmd+K command palette includes: pages, recent items, quick actions

## 11.3 Command Palette (Cmd+K)

Command palette power users साठी आहे — Cmd+K दाबल्यावर fuzzy search box उघडतो. User त्यात pages, recent items, quick actions search करू शकतो. हे feature velocity boost करते — mouse शिवाय सर्व navigation possible होते. Implementation cmdk library वर based आहे:

```

Cmd+K pressed
↓
Modal opens with search input
↓ (user types)
Fuzzy search across:
- All pages (filtered by permissions)
- Recent students (last 10 viewed)
- Recent invoices (last 10 created)
- Quick actions (Create Student, Generate Invoice)

```

```

- Settings pages
  ↓ (user selects)
Navigate to selected item OR execute action
  ↓
Modal closes

```

```

// Keyboard shortcuts:
// Cmd+K   Open palette
// Esc     Close
// ↑↓      Navigate results
// Enter   Select
// Cmd+Enter Open in new tab

```

## 12. Forms

Forms हे enterprise applications चा core interaction mechanism आहे. PreOne मध्ये React Hook Form + Zod combination use होतो — performance (uncontrolled refs) + type safety (schema-driven) दोन्ही. प्रत्येक form ची Zod schema आहे जी client + server validation चा single source of truth आहे.

### 12.1 Form Patterns

| Pattern                 | Implementation                                                    | Use Case                        |
|-------------------------|-------------------------------------------------------------------|---------------------------------|
| Controlled Components   | <Input {...register('name')} />                                   | Default for simple inputs       |
| Uncontrolled Components | useController for complex widgets (DatePicker, FileUpload)        | Performance optimization        |
| Field Arrays            | useFieldArray for repeating groups (guardians, education history) | Dynamic add/remove rows         |
| Conditional Fields      | watch() + conditional render                                      | Show/hide based on other fields |
| Async Validation        | Trigger on blur; debounce 500ms; show spinner                     | Username/email uniqueness       |
| Multi-step Forms        | useForm across steps; validate per step; persist to Zustand       | Wizard patterns (admission)     |
| Form Context            | FormProvider for deeply nested fields                             | Avoid prop drilling             |

| Pattern             | Implementation                                            | Use Case              |
|---------------------|-----------------------------------------------------------|-----------------------|
| Schema-driven Forms | React Hook Form + Zod resolver — UI generated from schema | Consistent validation |

## 12.2 Form Submission Flow

Form submission चा complete flow खालीलप्रमाणे आहे. प्रत्येक step specific responsibility handle करते — error कोणत्याही step पर येऊ शकतो आणि appropriate UI feedback शी translate होतो:

```

User fills form (controlled by React Hook Form)
↓ on submit
Zod resolver validates (client-side)
↓ valid
Submit handler calls API service
↓
API validates server-side (defense in depth)
↓ success
Invalidate relevant TanStack Query (refetch list)
↓
Show success toast
↓
Navigate to detail page OR reset form
-- OR -- on error
Show error toast with field-level details (if validation)
↓
setFormError() for programmatic field errors

```

## 12.3 Form Rules

- Every form has a Zod schema — single source of truth
- Submit button disabled during mutation (useFormStatus)
- Optimistic UI for non-critical mutations (mark attendance)
- Pessimistic UI for critical mutations (payment, delete)
- Confirm dialog before destructive actions
- Unsaved changes warning via beforeunload + route block
- Autosave for long forms (debounced 2s, save to draft endpoint)
- Reset form after successful create (not update)
- Server validation errors mapped to fields via setError()
- Display field-level error below input; show error count in submit button

## 12.4 Example — CreateStudentForm

खालील example complete form implementation दाखवते — Zod schema, React Hook Form integration, mutation hook, error handling. हे pattern सर्व forms साठी template आहे:

```
// modules/student/forms/create-student-form.tsx
'use client';

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { useCreateStudent } from '../hooks';
import { CreateStudentSchema } from '../validation';
import type { CreateStudentInput } from '../types';

export function CreateStudentForm({ onSuccess }: { onSuccess: () => void }) {
  {
    const {
      register,
      handleSubmit,
      formState: { errors, isSubmitting },
      setError,
    } = useForm<CreateStudentInput>({
      resolver: zodResolver(CreateStudentSchema),
    });

    const createStudent = useCreateStudent();

    const onSubmit = async (data: CreateStudentInput) => {
      try {
        await createStudent.mutateAsync(data);
        toast.success('Student created successfully');
        onSuccess();
      } catch (error) {
        if (error instanceof ApiError && error.code === 'VALIDATION_ERROR') {
          error.details.forEach(({ field, message }) => {
            setError(field, { message });
          });
        } else {
          toast.error('Failed to create student');
        }
      }
    };

    return (
      <form onSubmit={handleSubmit(onSubmit)}>
        <Input
          label="First Name"
          error={errors.firstName?.message}
          {...register('firstName')}
        />
      </form>
    );
  }
}
```

```

    />
    { /* ... more fields */ }
    <Button type="submit" loading={isSubmitting}>
      Create Student
    </Button>
  </form>
);
}

```

## 13. Validation

Validation हे defense-in-depth strategy आहे — same as backend. PreOne मध्ये 4 validation layers आहेत — client, business, API, server. प्रत्येक layer वेगवेगळ्या type ची invalid input catch करते. यामुळे invalid data कधीच database पर्यंत पोहोचत नाही.

### 13.1 Validation Levels

| Level               | Where                                          | Purpose                              | Examples                             |
|---------------------|------------------------------------------------|--------------------------------------|--------------------------------------|
| Client Validation   | Browser native + Zod schema in form            | Instant feedback; UX-friendly        | Required, max length, email format   |
| Business Validation | Cross-field rules in Zod refinements           | Catch logic errors before API call   | EndDate > StartDate, FeeMin < FeeMax |
| API Validation      | Backend DTO validation (class-validator + Zod) | Defense in depth; never trust client | Duplicate email, invalid references  |
| Server Validation   | Domain aggregate invariants + DB constraints   | Last line of defense                 | Unique roll number, section capacity |

### 13.2 Validation Examples

खालील table common field validation rules दाखवते. हे rules Zod schema मध्ये implement होतात आणि frontend + backend दोन्ही ठिकाणी enforce होतात:

| Field        | Rules                                  | Implementation                |
|--------------|----------------------------------------|-------------------------------|
| Student Name | Required, Max 100, Trim, No Emoji      | @Matches(/^[\p{L}\s.'-]+\$/u) |
| Email        | Required, RFC 5322, Max 254, Lowercase | @IsEmail()                    |



| Field         | Rules                                        | Implementation                       |
|---------------|----------------------------------------------|--------------------------------------|
| Phone (India) | Required, 10 digits, starts 6-9              | @Matches(/^([6-9]\d{9})\$/)          |
| Date of Birth | Required, ISO-8601, age 2-6 for preschool    | @IsDateString() + custom             |
| Aadhaar       | Optional, 12 digits, Verhoeff checksum valid | Custom validator                     |
| PAN           | Optional, 10 chars, ABCDE1234F format        | @Matches(/[A-Z]{5}[0-9]{4}[A-Z]{1}/) |
| Fee Amount    | Required, positive integer (paise), max 10L  | @Min(1) @Max(10000000)               |
| Date Range    | EndDate >= StartDate, max 365 days span      | Zod .refine()                        |

### 13.3 Zod Schema Example

Zod schema हे single source of truth आहे — याच schema पासून TypeScript types infer होतात, frontend form validation होते, आणि backend DTO validation होते. यामुळे type drift prevent होते:

```
// modules/student/validation/create-student-schema.ts
import { z } from 'zod';

export const CreateStudentSchema = z.object({
  firstName: z.string().min(1).max(100).trim().regex(/^[p{L}s\.-]+$/u, 'Invalid name'),
  lastName: z.string().min(1).max(100).trim().regex(/^[p{L}s\.-]+$/u, 'Invalid name'),
  dateOfBirth: z.string().datetime().refine((dob) => {
    const age = computeAge(dob);
    return age >= 2 && age <= 6; // preschool age range
  }, 'Age must be between 2 and 6 years for preschool'),
  gender: z.enum(['MALE', 'FEMALE', 'OTHER']),
  address: AddressSchema,
  guardians: z.array(GuardianSchema).min(1, 'At least one guardian required'),
  aadhaar: z.string().optional().refine(validateAadhaarVerhoeff, 'Invalid Aadhaar'),
});

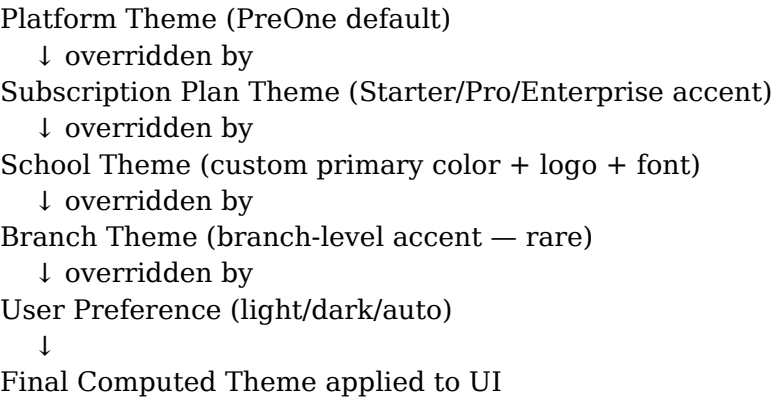
// Type inferred from schema — used in form + API service
export type CreateStudentInput = z.infer<typeof CreateStudentSchema>;
```

## 14. Theme Engine

Theme Engine हे PreOne च्या white-label strategy चा foundation आहे. PreOne सारख्या multi-tenant SaaS मध्ये प्रत्येक school चा own brand identity आहे — logo, colors, fonts. Theme Engine याला runtime पर support करतो — कोणतेही redeploy न करता school switch केल्यावर theme change होते. हे UI Design Philosophy मधील Theme Engine शी पूर्णपणे aligned असेल.

### 14.1 Theme Hierarchy

Theme resolution चा order खालीलप्रमाणे आहे — Platform → Subscription → School → Branch → User. Outer themes baseline provide करतात, inner themes override करतात. User preference (light/dark) सर्वात inner layer आहे — ते सर्वात जास्त priority ला:



### 14.2 Theme Hierarchy Layers

| Layer                   | Scope                                                   | Override Rule                       |
|-------------------------|---------------------------------------------------------|-------------------------------------|
| Platform Theme          | PreOne default brand theme                              | Baseline — applied when no override |
| Subscription Plan Theme | Tier-based (Starter, Pro, Enterprise) accent variations | Subtle accent shifts                |
| School Theme            | Per-tenant custom primary color + logo                  | Loaded on login, persisted          |
| Branch Theme            | Branch-level accent (optional, rare)                    | Multi-branch schools                |
| User Preference         | Light / Dark / Auto mode toggle                         | Per-user, persisted                 |

## 14.3 Theme Tokens

Theme tokens हे CSS custom properties (variables) आहेत जे runtime पर swap होऊ शकतात. oklch() color space use होतो — wide gamut + perceptual uniformity. खालील table सर्व tokens ची catalog करते:

| Token      | Purpose                                             | Example Value                                               |
|------------|-----------------------------------------------------|-------------------------------------------------------------|
| Primary    | Brand primary color (buttons, links, active states) | oklch(0.55 0.18 240) — varies per school                    |
| Secondary  | Supporting brand color                              | oklch(0.70 0.10 200)                                        |
| Success    | Success feedback (paid, approved, completed)        | oklch(0.65 0.18 145)                                        |
| Warning    | Caution (pending, due soon, low stock)              | oklch(0.75 0.15 85)                                         |
| Danger     | Errors, destructive actions (delete, reject)        | oklch(0.60 0.20 25)                                         |
| Info       | Informational badges, tooltips                      | oklch(0.65 0.12 230)                                        |
| Background | App background                                      | Light: oklch(0.98 0.005 240);<br>Dark: oklch(0.18 0.01 240) |
| Surface    | Card / panel background                             | Light: oklch(1 0 0); Dark: oklch(0.22 0.01 240)             |
| Text       | Primary body text                                   | Light: oklch(0.20 0.02 240);<br>Dark: oklch(0.95 0.005 240) |
| Border     | Subtle dividers, input borders                      | Light: oklch(0.90 0.01 240);<br>Dark: oklch(0.30 0.01 240)  |

## 14.4 Theme Modes

PreOne मध्ये 5 theme modes आहेत. Light/Dark/Auto हे standard modes आहेत, High Contrast accessibility साठी, White Label enterprise clients साठी:

| Mode       | Description                            | Use Case              |
|------------|----------------------------------------|-----------------------|
| Light Mode | Default — bright background, dark text | Brightness preference |

| Mode          | Description                                                        | Use Case            |
|---------------|--------------------------------------------------------------------|---------------------|
| Dark Mode     | Inverted — dark background, light text                             | Low-light comfort   |
| Auto Mode     | Follows system preference via prefers-color-scheme                 | Best UX default     |
| High Contrast | Accessibility mode — maximum contrast, larger text                 | WCAG AAA compliance |
| White Label   | Full rebrand — different logo, colors, font for enterprise clients | Resale / franchise  |

## 14.5 Theme Application

Theme application चा mechanism खालीलप्रमाणे आहे. CSS variables runtime पर swap होतात — React re-render complete app ला होत नाही. हे performance optimal आहे. Theme provider फक्त `<html>` element च्या `data-theme` attribute ला update करतो आणि CSS variables automatically cascade होतात:

```
// User logs in
↓
Fetch school theme: GET /v1/theme
↓
Merge with platform defaults (deep merge)
↓
Compute final theme (apply user preference)
↓
Set CSS variables on :root via <style> tag
↓
All components re-render with new colors (CSS cascade)
```

```
// CSS:
:root {
  --color-primary: oklch(0.55 0.18 240);
  --color-background: oklch(0.98 0.005 240);
  /* ... */
}
```

```
[data-theme='dark'] {
  --color-background: oklch(0.18 0.01 240);
  --color-text: oklch(0.95 0.005 240);
  /* ... */
}
```

## 15. Component Library

Component Library हे PreOne च्या UI चा foundation आहे. PreOne मध्ये दोन component categories आहेत — Base Components (generic, reusable across modules) आणि Composite Components (domain-specific, used in specific modules). Base components ShadCN UI वर based आहेत — copy-into-repo model, fully owned, no version drift.

### 15.1 Base Components

Base components हे generic UI primitives आहेत — Button, Input, Card, etc. हे components कोणत्याही module मध्ये use होऊ शकतात. ShadCN UI पासून copied, fully themable, accessible by default. खालील table सर्व base components ची catalog करते:

| Component  | Variants / Features                                                             | Use Case       |
|------------|---------------------------------------------------------------------------------|----------------|
| Button     | Primary, Secondary, Ghost, Danger, Link variants; sizes sm/md/lg; loading state | All actions    |
| Input      | Text, password, number, email, search; with prefix/suffix icons; error state    | Form inputs    |
| Select     | Single/multi/async; searchable; grouping; with avatars                          | Dropdowns      |
| Textarea   | Auto-resize; character count; max length                                        | Long text      |
| Checkbox   | Indeterminate state; controlled; group with CheckAll                            | Multi-select   |
| Radio      | Group with cards; horizontal/vertical layout                                    | Single-select  |
| Switch     | Toggle state; loading; disabled; with label                                     | Binary toggle  |
| DatePicker | Single/range; time; locale-aware; shortcuts (today, this week)                  | Date selection |

| Component  | Variants / Features                                              | Use Case            |
|------------|------------------------------------------------------------------|---------------------|
| FileUpload | Drag-drop; multi-file; progress; preview; type validation        | Document upload     |
| Card       | Header/body/footer slots; hoverable; selectable                  | Content containers  |
| Table      | Built on TanStack Table; server pagination; sort; filter         | Data display        |
| Dialog     | Modal with backdrop; sizes sm/md/lg/xl; close on ESC             | Modal interactions  |
| Drawer     | Side panel; left/right; sizes; nested                            | Detail panels       |
| Tabs       | Underline/pills/cards variants; controlled/uncontrolled          | Section navigation  |
| Toast      | Stack; auto-dismiss; manual close; types success/error/warn/info | Notifications       |
| Badge      | Count, dot, label variants; colors per status                    | Status indicators   |
| Avatar     | Image/initials fallback; sizes; group overlap                    | User representation |
| Breadcrumb | Auto-generated from route; custom items; with icons              | Navigation trail    |

## 15.2 Composite Components

Composite components हे domain-specific complex components आहेत — StudentCard, AttendanceGrid, FeeSummaryCard, etc. हे components specific business context represent करतात आणि multiple base components compose करतात. खालील table PreOne चे composite components दाखवते:

| Component   | Composition                                                   | Used In               |
|-------------|---------------------------------------------------------------|-----------------------|
| StudentCard | Student photo, name, class, section, attendance %, fee status | List view + dashboard |

| Component            | Composition                                                               | Used In              |
|----------------------|---------------------------------------------------------------------------|----------------------|
| AttendanceGrid       | Monthly grid with color-coded status, click-to-edit, legend               | Attendance module    |
| FeeSummaryCard       | Total/paid/outstanding amounts, last payment, next due                    | Finance dashboard    |
| ObservationTimeline  | Vertical timeline of teacher observations with milestones                 | Academics module     |
| DashboardKpiTile     | KPI label, value, delta vs last period, sparkline                         | All dashboards       |
| AdmissionPipeline    | Funnel visualization: Leads → Applied → Counselling → Approved → Enrolled | Admissions dashboard |
| InvoicePreview       | Print-ready invoice layout with itemized breakdown                        | Finance module       |
| ReportCardPreview    | Term report card with grades, teacher remarks, signature                  | Academics module     |
| StaffLeaveCalendar   | Calendar view of staff leaves with color coding per type                  | HR module            |
| InventoryStatusBoard | Low-stock alerts, reorder suggestions, recent movements                   | Inventory dashboard  |

## 15.3 Component Rules

- Base components live in components/ui/ — copied from ShadCN, fully owned
- Composite components live in modules/{module}/components/ — domain-specific
- Components NEVER call API services directly — only via hooks
- Components NEVER access global state directly — props or hooks
- Components must accept className for Tailwind composition
- Components must forward refs via React.forwardRef
- Components must be accessible (ARIA, keyboard, focus management)
- Every component has a Storybook story with variants

- Breaking changes to base components require version bump + migration guide
- Composite components document their data dependencies in JSDoc

## 15.4 Storybook Integration

प्रत्येक component ची Storybook story आहे — variants, states, edge cases document केलेले. Storybook हे living documentation आहे — designer + developer + QA सर्व एकच reference बघतात. Story file component च्या शेजारी co-located आहे (student-card.stories.tsx). Chromatic integration visual regression catch करतो:

```
// components/ui/button/button.stories.tsx
import type { Meta, StoryObj } from '@storybook/react';
import { Button } from './button';

const meta = {
  title: 'UI/Button',
  component: Button,
  parameters: { layout: 'centered' },
  tags: ['autodocs'],
  argTypes: {
    variant: { control: 'select', options: ['primary', 'secondary', 'ghost', 'danger', 'link'] },
    size: { control: 'select', options: ['sm', 'md', 'lg'] },
  },
} satisfies Meta<typeof Button>;

export default meta;
type Story = StoryObj<typeof meta>;

export const Primary: Story = {
  args: { variant: 'primary', size: 'md', children: 'Click me' },
};

export const Loading: Story = {
  args: { variant: 'primary', loading: true, children: 'Submit' },
};

export const Disabled: Story = {
  args: { variant: 'primary', disabled: true, children: 'Submit' },
};
```

## 16. Data Tables

Data Tables हे enterprise applications चा core UI element आहे — students list, invoices, attendance, staff. PreOne मध्ये TanStack Table (headless) + ShadCN styling च combination



use होतो. Headless approach देतो full styling control, TanStack Table देतो enterprise-grade features (sorting, filtering, pagination, virtualization).

## 16.1 Table Features

PreOne च्या data tables मध्ये खालील features standard आहेत. प्रत्येक feature ADR-FA-015 मध्ये documented आहे — PR review मध्ये verify केले जाते की new table यापैकी सर्व features support करते:

| Feature             | Implementation                                           | Purpose                         |
|---------------------|----------------------------------------------------------|---------------------------------|
| Server Pagination   | Cursor-based for large datasets; page-based for small    | Avoid loading all rows          |
| Column Filter       | Per-column filter UI (text, select, date range)          | Combined with global search     |
| Global Search       | Fuzzy search across visible columns; debounced 300ms     | Quick filter UX                 |
| Sorting             | Single or multi-column; server-side; indicator in header | Click header to toggle          |
| Export              | Excel, CSV, PDF — server-generated via BullMQ job        | Respects current filter         |
| Bulk Actions        | Row selection via checkbox; action bar appears           | Bulk delete, bulk status change |
| Column Visibility   | User can hide/show columns; persisted per user           | Personalization                 |
| Sticky Header       | Header stays visible on vertical scroll                  | Long tables                     |
| Sticky First Column | Name column stays visible on horizontal scroll           | Wide tables                     |
| Row Expansion       | Click row to expand details (no navigation)              | Quick preview                   |
| Empty State         | Friendly illustration + CTA when no data                 | Better than blank               |
| Loading State       | Skeleton rows during fetch (not spinner)                 | Perceived performance           |

| Feature        | Implementation                                              | Purpose              |
|----------------|-------------------------------------------------------------|----------------------|
| Error State    | Inline error + retry button                                 | Graceful degradation |
| Density Toggle | Compact / Regular / Comfortable row height                  | User preference      |
| Responsive     | Horizontal scroll on mobile; columns hide based on priority | Mobile-friendly      |

## 16.2 Table Rules

- All tables built on TanStack Table (headless) + ShadCN styling
- Server-side pagination/sorting/filtering — never client-side for large data
- URL state for filters (shareable + back-button friendly)
- Debounce text filters 300ms to avoid excessive API calls
- Bulk actions require explicit confirm dialog
- Export runs async via BullMQ — email link when ready
- Column visibility persisted per user per table
- Row click navigates to detail (unless expansion configured)
- Keyboard navigation: arrow keys + Enter to select
- Virtual scrolling for 1000+ rows (TanStack Virtual)

## 16.3 Table Component Example

खालील example दाखवते कसे DataTable component use होतो. हे component headless TanStack Table चा wrapper आहे — consumer फक्त columns + data + fetch function provide करतो. Server-side pagination, sorting, filtering built-in:

```
// modules/student/components/student-table.tsx
'use client';

import { DataTable } from '@components/ui/data-table';
import { useStudents } from '../hooks';
import { columns } from './student-columns';

export function StudentTable({ filters }: { filters: StudentFilters }) {
  const [pagination, setPagination] = usePagination();
  const [sorting, setSorting] = useSorting();
  const [search, setSearch] = useDebounce("", 300);
```

```

const { data, isLoading } = useStudents({
  ...filters,
  page: pagination.pageIndex,
  pageSize: pagination.pageSize,
  sort: sorting,
  search,
});

return (
  <DataTable
    columns={columns}
    data={data?.items ?? []}
    loading={isLoading}
    pagination={pagination}
    onPageChange={setPagination}
    sorting={sorting}
    onSortingChange={setSorting}
    search={search}
    onSearchChange={setSearch}
    totalCount={data?.totalCount ?? 0}
    bulkActions={[
      { label: 'Delete Selected', onClick: handleBulkDelete, permission:
'STUDENT_DELETE' },
      { label: 'Export Selected', onClick: handleBulkExport, permission:
'STUDENT_EXPORT' },
    ]}
    emptyState={<EmptyState title="No students"
action={<CreateStudentButton /> } />}
  />
);
}

```

## 17. Charts

Charts हे data visualization चा mechanism आहे — dashboards, reports, analytics. PreOne मध्ये Recharts चा use होतो — composable API, theme-aware, responsive. सर्व charts theme tokens use करतात — school theme change झाल्यावर charts आपोआप update होतात.

### 17.1 Chart Types

PreOne मध्ये 10 chart types use होतात — line, bar, stacked bar, pie/donut, area, scatter, heatmap, funnel, gauge, sparkline. प्रत्येक chart type ची specific use case आहे:

| Chart       | Use Case                                                    | Example                |
|-------------|-------------------------------------------------------------|------------------------|
| Line Chart  | Trends over time — fee collection, attendance %, enrollment | Time-series data       |
| Bar Chart   | Comparisons — branch-wise revenue, class-wise strength      | Categorical comparison |
| Stacked Bar | Composition — fee heads, attendance reasons                 | Part-to-whole          |
| Pie / Donut | Distribution — admission sources, payment modes             | ≤ 6 categories         |
| Area Chart  | Cumulative trends — revenue YTD, expenses                   | Cumulative             |
| Scatter     | Correlation — age vs. fee, attendance vs. grades            | Two-variable           |
| Heatmap     | Density — attendance patterns, classroom utilization        | Grid data              |
| Funnel      | Pipeline — admission stages, conversion                     | Sequential stages      |
| Gauge       | Single KPI — target achievement %                           | Single metric          |
| Sparkline   | Inline mini chart in tables/cards                           | Compact trend          |

## 17.2 Chart Rules

- All charts via Recharts (composable, theme-aware)
- Colors pulled from theme tokens (consistent with brand)
- Tooltips show precise values + date on hover
- Legends positioned below chart; collapsible
- Responsive container; min-height to prevent layout shift
- Loading state: skeleton chart (not spinner)
- Empty state: friendly message + CTA
- Data labels disabled by default (enable on user toggle)

- Accessibility: data table alternative behind toggle
- Export to PNG + SVG via chart wrapper utility

### 17.3 Chart Example

खालील example दाखवते कसे theme-aware chart component तयार करायचे. Colors theme tokens पासून येतात — hardcoded colors नाहीत. Responsive container layout shift prevent करतो:

```
// components/charts/fee-collection-chart.tsx
'use client';

import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip,
  ResponsiveContainer } from 'recharts';
import { useTheme } from '@providers/theme-provider';

export function FeeCollectionChart({ data }: { data: FeePoint[] }) {
  const { theme } = useTheme();

  return (
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={data}>
        <CartesianGrid strokeDasharray="3 3" stroke={theme.border} />
        <XAxis dataKey="month" stroke={theme.text} />
        <YAxis stroke={theme.text} tickFormatter={(v) =>
formatCurrency(v)} />
        <Tooltip
          contentStyle={{ backgroundColor: theme.surface, border: `1px solid $
{theme.border}` }}
          formatter={(value: number) => [formatCurrency(value), 'Collected']}
        />
        <Line
          type="monotone"
          dataKey="collected"
          stroke={theme.primary}
          strokeWidth={2}
          dot={{ fill: theme.primary }}
        />
      </LineChart>
    </ResponsiveContainer>
  );
}
```

## 18. Notifications

Notifications हे user feedback चा mechanism आहे — action confirmations, errors, system alerts. PreOne मध्ये 8 notification types आहेत — toast, banner, modal, inline, push (browser

+ mobile), badge counter, realtime indicator. प्रत्येक type ची specific use case आहे — wrong type use केल्यास UX suffers.

## 18.1 Notification Types

| Type               | Description                                 | Use Case                                      | Persistence                |
|--------------------|---------------------------------------------|-----------------------------------------------|----------------------------|
| Toast              | Brief, auto-dismiss (3-5s), top-right stack | Success/error after action                    | Transient                  |
| Banner             | Full-width, dismissible, above content      | System-wide announcements, maintenance        | Persistent until dismissed |
| Modal              | Centered dialog, requires interaction       | Critical confirmations (delete, irreversible) | Blocking                   |
| Inline             | Within form/page, contextual to section     | Field-level hints, contextual help            | Contextual                 |
| Push (Browser)     | OS-level notification when tab inactive     | New message, fee due reminder                 | Opt-in                     |
| Push (Mobile)      | FCM/APNs for mobile app (future)            | Critical alerts, chat messages                | Future scope               |
| Badge Counter      | Numeric indicator on icon/menu              | Unread count, pending approvals               | Persistent                 |
| Realtime Indicator | Pulse animation on live data                | Attendance being marked, payment received     | Transient                  |

## 18.2 Notification Rules

- Max 3 toasts visible simultaneously — older ones queue
- Toasts auto-dismiss after 5s (success) or 10s (error)
- Error toasts include 'View Details' link for full error
- Banners persist across page navigation (until dismissed)
- Realtime notifications via Socket.IO — instant badge update
- Push notifications require explicit user opt-in
- Notifications respect Do Not Disturb (user-configurable hours)
- Notification preferences stored per-user, synced across devices

- Critical notifications (fee due, attendance alert) bypass DND
- Audit log for sent notifications (compliance requirement)

### 18.3 Toast System

Toast system हे सर्वात common notification mechanism आहे. PreOne मध्ये Sonner library use होते — stack-based, auto-dismiss, swipe-to-dismiss. Toasts top-right corner मध्ये stack होतात, max 3 visible simultaneously. Older ones queue. Error toasts जास्त दिसतात (10s vs 5s) आणि 'View Details' link include करतात:

```
// lib/toast.ts
import { toast } from 'sonner';

export const showToast = {
  success: (message: string, description?: string) =>
    toast.success(message, { description, duration: 5000 }),

  error: (message: string, options?: { description?: string; traceId?: string })
    =>
    toast.error(message, {
      description: options?.description,
      duration: 10000, // longer for errors
      action: options?.traceId
        ? { label: 'View Details', onClick: () => copyTraceId(options.traceId!) }
        : undefined,
    }),

  warning: (message: string, description?: string) =>
    toast.warning(message, { description, duration: 7000 }),

  info: (message: string, description?: string) =>
    toast.info(message, { description, duration: 5000 }),

  loading: (message: string) => toast.loading(message),
  promise: toast.promise,
};
```

### 19. File Upload

File Upload हे document-heavy preschool operations चा core feature आहे — student documents, photos, invoices, certificates. PreOne मध्ये pre-signed URL pattern use होतो — client directly uploads to S3 (bypassing backend), backend फक्त metadata persist करतो. यामुळे backend load कमी होते आणि upload speed वाढते.

## 19.1 Upload Features

| Feature          | Implementation                                          | Benefit            |
|------------------|---------------------------------------------------------|--------------------|
| Drag & Drop      | Drop zone with hover state; multi-file                  | UX optimization    |
| Progress Bar     | Per-file progress; abort button                         | Long uploads       |
| Preview          | Image/video preview; PDF first page; icon for others    | Before commit      |
| Type Validation  | MIME + extension check; reject invalid types            | Security           |
| Size Validation  | Per-file max (10MB); per-session max (100MB)            | Storage limits     |
| Chunked Upload   | Large files (>50MB) uploaded in 5MB chunks; resumable   | Reliability        |
| Virus Scan       | ClamAV scan on backend before commit to S3              | Security           |
| Compression      | Client-side image compression (max 1920px, 80% quality) | Bandwidth savings  |
| Pre-signed URL   | Direct upload to S3 via pre-signed URL (bypass backend) | Performance        |
| Retry on Failure | Auto-retry 3x with exponential backoff                  | Network resilience |

## 19.2 Upload Flow

Complete upload flow खालीलप्रमाणे आहे. Pre-signed URL pattern backend load कमी करतो — file data backend through जात नाही, थेट S3 ला upload होते. Backend फक्त metadata persist करतो. Virus scan backend पर happen होतो — ClamAV integration:

```

User selects files (drag-drop or click)
↓
Client-side validation (type, size)
↓
Request pre-signed URL from backend
↓
Upload directly to S3 via pre-signed URL (with progress)
↓

```



```

Notify backend of upload completion (POST /uploads/confirm)
↓
Backend virus scan + persist metadata
↓
Return file ID + URL to client
↓
Form submission includes file IDs (not files)

```

### 19.3 FileUpload Component Example

FileUpload component चा implementation खालीलप्रमाणे आहे. Drag-drop, progress, preview, validation — सर्व features built-in. Component fully controlled — parent form state through manage होते:

```

// components/ui/file-upload/file-upload.tsx
'use client';

interface FileUploadProps {
  accept: string[];      // e.g., ['image/*', 'application/pdf']
  maxSize: number;       // bytes
  multiple?: boolean;
  onUpload: (files: UploadedFile[]) => void;
  folder: string;        // S3 folder (e.g., 'students/abc-123/documents')
}

export function FileUpload({ accept, maxSize, multiple, onUpload, folder }:
FileUploadProps) {
  const [uploads, setUploads] = useState<UploadState[]>([]);

  const handleFiles = async (files: File[]) => {
    for (const file of files) {
      // Validate
      if (file.size > maxSize) {
        showToast.error(`File too large: ${file.name}`);
        continue;
      }

      // Get pre-signed URL
      const { uploadUrl, fileId } = await getUploadUrl({ filename: file.name,
folder });

      // Upload directly to S3
      await axios.put(uploadUrl, file, {
        onUploadProgress: (progress) => {
          setUploads((prev) => updateProgress(prev, fileId, progress));
        },
      });
    }
  };
}

```

```

    // Confirm with backend
    await confirmUpload({ fileId });
  }
};

return (
  <Dropzone accept={accept} multiple={multiple} onDrop={handleFiles}>
    {uploads.map((u) => <UploadProgress key={u.fileId} upload={u} />)}
  </Dropzone>
);
}

```

## 20. Error Handling

Error Handling हे user experience चा critical part आहे — कसे errors present केले जाते ते user frustration कमी करते किंवा वाढवते. PreOne मध्ये layered error handling आहे — global error boundary, route-level error boundary, component-level error fallback, आणि form-level error display. प्रत्येक layer different granularity पर handle करते.

### 20.1 Error Types

PreOne मध्ये 8 primary error types आहेत. प्रत्येक type ची specific UI treatment आहे — validation errors inline, network errors persistent banner, critical errors modal:

| Error             | HTTP Status | UI Treatment                                                         | Recovery               |
|-------------------|-------------|----------------------------------------------------------------------|------------------------|
| Validation Error  | 422         | Field-level errors displayed inline below inputs                     | setFormError() + toast |
| Permission Denied | 403         | Inline 403 fallback component or redirect to /forbidden              | <Can> fallback         |
| Network Error     | —           | Offline banner + retry button; queue mutations if offline            | useOnlineStatus hook   |
| Session Expired   | 401         | Silent refresh attempt; if fails, redirect to /login with return URL | Axios interceptor      |
| Server Error      | 5xx         | Toast + error ID for support; retry button                           | Global error boundary  |

| Error        | HTTP Status | UI Treatment                                  | Recovery          |
|--------------|-------------|-----------------------------------------------|-------------------|
| Not Found    | 404         | Friendly 404 page with navigation suggestions | not-found.tsx     |
| Rate Limited | 429         | Toast with retry-after countdown              | Axios interceptor |
| Conflict     | 409         | Toast with conflict details; refresh data     | Refetch + toast   |

## 20.2 Error Flow

Error handling चा complete flow खालीलप्रमाणे आहे. खालील flow ensure करते की user ला कधीच raw stack trace दिसत नाही, आणि त्याला clear recovery path मिळतो:

```

API Error occurs (interceptor catches)
↓
Normalize to ApiError shape
↓
If 401: attempt silent refresh
  ↓ refresh success
  Retry original request
  -- OR -- refresh fail
  Redirect to /login
  ↓ (other errors)
TanStack Query onError callback
↓
Show toast (transient) OR modal (critical)
↓
If retryable, show retry button
↓
Fallback UI if component crashed (Error Boundary)

```

## 20.3 Error Handling Rules

- Every route has error.tsx boundary catching unexpected errors
- Global error boundary in app/global-error.tsx catches root errors
- Form validation errors shown inline (not toast)
- Network errors show persistent banner (not toast)
- Critical errors (5xx) include error ID for support tickets
- Error states never expose stack traces to users
- Retry button on all transient errors

- 404 page includes search + navigation suggestions
- 403 page explains why access denied + link to request access
- Errors logged to Sentry/LogRocket for analysis

## 20.4 Global Error Boundary

Global error boundary हे root-level error catcher आहे — React tree मध्ये कोणताही uncaught error यामध्ये handle होतो. app/global-error.tsx Next.js convention follow करतो. खालील example दाखवते implementation:

```
// app/global-error.tsx
'use client';

import { useEffect } from 'react';
import * as Sentry from '@sentry/nextjs';

export default function GlobalError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  useEffect(() => {
    Sentry.captureException(error);
    console.error('Global error:', error);
  }, [error]);

  return (
    <html>
      <body>
        <ErrorLayout
          title="Something went wrong"
          description="An unexpected error occurred. Our team has been notified."
          errorId={error.digest}
          actions={(
            <>
              <Button onClick={reset}>Try again</Button>
              <Button variant="ghost" onClick={() => window.location.href =
'/dashboard'}>
                Go to Dashboard
              </Button>
            </>
          )}
        />
      </body>
```

```

    </html>
  );
}

```

## 21. Performance

Performance हे user experience चा direct contributor आहे — Google च्या research नुसार 100ms साठी wait करणारे users 7% conversion कमी करतात. PreOne चे SLO (Service Level Objective) Core Web Vitals शी aligned आहेत — LCP < 2.5s, INP < 200ms, CLS < 0.1. Continuous monitoring Lighthouse CI + Vercel Analytics through होते.

### 21.1 Performance Tactics

PreOne मध्ये 12 distinct performance tactics apply होतात. प्रत्येक tactic ची specific use case आहे — blanket application नाही. खालील table सर्व tactics ची catalog करते:

| Tactic                   | Mechanism                                              | Benefit                |
|--------------------------|--------------------------------------------------------|------------------------|
| Code Splitting           | next/dynamic for heavy components (charts, editors)    | Smaller initial bundle |
| Route-based Lazy Loading | App Router auto-codesplits per route                   | Faster navigation      |
| Image Optimization       | next/image with WebP/AVIF; lazy load; blur placeholder | 60% smaller images     |
| Virtual Scrolling        | TanStack Virtual for 1000+ rows                        | Smooth scrolling       |
| Memoization              | React.memo, useMemo, useCallback strategically         | Avoid re-renders       |
| Suspense                 | Streaming SSR with React Suspense boundaries           | Faster TTFB            |
| Partial Hydration        | Server Components + selective 'use client'             | Less JS to client      |
| Prefetch                 | Link prefetch on hover; route prefetch on viewport     | Instant navigation     |
| Cache Headers            | Static assets immutable; HTML revalidate               | CDN efficiency         |
| Bundle Analysis          | next/bundle-analyzer in CI; budget alerts              | Prevent bloat          |

| Tactic          | Mechanism                                    | Benefit          |
|-----------------|----------------------------------------------|------------------|
| Tree Shaking    | ESM imports; named imports only              | Smaller bundles  |
| Font Subsetting | next/font with subset 'latin' + 'devanagari' | Faster font load |

## 21.2 Performance SLOs

SLOs measurable performance targets आहेत. या targets consistently miss झाल्यास performance optimization sprint triggered होतो. PreOne चे SLOs aggressive पण achievable आहेत — industry benchmarks शी aligned:

| Metric                          | Target                      | Scope                         |
|---------------------------------|-----------------------------|-------------------------------|
| LCP (Largest Contentful Paint)  | < 2.5s                      | Core Web Vital                |
| INP (Interaction to Next Paint) | < 200ms                     | Core Web Vital (replaces FID) |
| CLS (Cumulative Layout Shift)   | < 0.1                       | Core Web Vital                |
| TTFB (Time to First Byte)       | < 800ms                     | Server response               |
| Route Transition (cached)       | < 100ms                     | SPA navigation                |
| Route Transition (uncached)     | < 500ms                     | First visit                   |
| Bundle Size (initial)           | < 200 KB gzipped            | Landing page                  |
| Bundle Size (per route)         | < 100 KB gzipped additional | Lazy-loaded                   |
| API Response (cached)           | < 50ms                      | TanStack Query hit            |
| API Response (network)          | < 500ms p95                 | Backend SLO                   |

## 21.3 Bundle Analysis

Bundle size हे performance चा critical factor आहे — initial bundle 越大, TTFB + parse time जास्त. PreOne मध्ये next/bundle-analyzer CI मध्ये run होतो — budget exceed झाल्यास build fail. खालील example दाखवते bundle structure आणि budgets:

Bundle Budget:

- Initial JS (gzip): < 200 KB
- Initial CSS (gzip): < 30 KB
- Per-route JS (gzip): < 100 KB additional
- Total (lazy): < 1 MB

Composition (example):

|                       |                         |
|-----------------------|-------------------------|
| next/ (framework)     | ~80 KB                  |
| react + react-dom     | ~45 KB                  |
| @tanstack/react-query | ~12 KB                  |
| zustand               | ~3 KB                   |
| react-hook-form       | ~9 KB                   |
| zod                   | ~12 KB                  |
| axios                 | ~12 KB                  |
| recharts (lazy)       | ~45 KB (route-split)    |
| @tanstack/react-table | ~20 KB (lazy)           |
| -----                 |                         |
| Initial total         | ~173 KB (within budget) |

Monitoring:

- CI runs next build --analyze
- Bundle sizes stored per build
- PR shows bundle diff vs main
- Alert if any chunk grows > 10%

## 22. Accessibility

Accessibility (a11y) हे PreOne चा non-negotiable requirement आहे — WCAG 2.2 AA compliance. Section 508, ADA, आणि India च्या Rights of Persons with Disabilities Act 2016 सर्व compliance mandate करतात. PreOne मध्ये a11y by-default आहे — components accessible-by-construction, automated testing CI मध्ये, manual screen reader testing release cycle मध्ये.

### 22.1 Accessibility Rules

खालील rules WCAG 2.2 AA compliance सुनिश्चित करतात. यातील बऱ्याच rules automated axe-core tests ने enforce होतात — बाकी PR review मध्ये check होतात:

- WCAG 2.2 AA compliance (target AAA where feasible)
- Semantic HTML (nav, main, section, article, aside)
- ARIA labels only when semantic HTML insufficient
- Keyboard navigation for all interactions (Tab, Enter, ESC, arrows)
- Focus management: visible focus ring, logical tab order, focus trap in modals
- Color contrast: 4.5:1 for normal text, 3:1 for large text
- Never convey information by color alone (add icon + text)
- Alt text for all meaningful images; empty alt for decorative
- Form labels associated via htmlFor; error messages via aria-describedby

- Skip-to-content link at top of page
- Live regions for dynamic updates (aria-live)
- Reduced motion support via prefers-reduced-motion
- Screen reader testing with NVDA + VoiceOver in CI
- Automated a11y tests via axe-core in Vitest + Playwright

## 22.2 Automated Testing

Automated a11y testing हे CI pipeline चा mandatory part आहे — PR merge होण्यापूर्वी zero violations पाहिजे. axe-core Vitest integration सह component tests मध्ये run होते, Playwright integration सह E2E tests मध्ये run होते:

```
// Example: a11y test in component test
import { render } from '@testing-library/react';
import { axe } from 'vitest-axe';
import { Button } from './button';

describe('Button a11y', () => {
  it('should have no violations', async () => {
    const { container } = render(<Button>Click me</Button>);
    const results = await axe(container);
    expect(results).toHaveNoViolations();
  });

  it('should have no violations when disabled', async () => {
    const { container } = render(<Button disabled>Click me</Button>);
    const results = await axe(container);
    expect(results).toHaveNoViolations();
  });
});

// CI: GitHub Actions runs all a11y tests on every PR
// - Failure blocks merge
// - Report uploaded as PR comment with violation details
```

## 23. Internationalization

Internationalization (i18n) हे PreOne च्या India-first strategy चा core आहे — 3 locales launch साठी (en-IN, hi, mr). India मध्ये preschool education मराठी, हिंदी, आणि English मध्ये happen होते — users ची preferred language vary होते. next-intl library App Router native support देते — type-safe message keys, ICU MessageFormat, locale routing.



## 23.1 i18n Configuration

| Aspect             | Value                                                      | Notes                                        |
|--------------------|------------------------------------------------------------|----------------------------------------------|
| Supported Locales  | en-IN (default), hi, mr                                    | 3 locales at launch                          |
| Locale Routing     | /en/dashboard,<br>/hi/dashboard,<br>/mr/dashboard          | SEO + UX                                     |
| Message Format     | next-intl with ICU<br>MessageFormat                        | Plurals, gender, dates                       |
| Translation Source | Crowdin (or Lokalise) —<br>managed translation<br>platform | Translator workflow                          |
| Fallback           | en-IN for missing keys (never<br>show key string)          | Graceful degradation                         |
| Date/Time          | Intl.DateTimeFormat with<br>locale                         | e.g., 14/07/2026 (en-IN),<br>१४/०७/२०२६ (hi) |
| Numbers            | Intl.NumberFormat with<br>locale + currency                | ₹1,234.56 (en-IN), १,२३४.५६<br>(mr)          |
| RTL Support        | Architecture-ready<br>(Arabic/Urdu future)                 | Logical properties                           |

## 23.2 i18n Rules

- All user-facing strings in messages/{locale}.json — never hardcoded
- Translation keys namespaced by module: student.create.title
- Plurals handled via ICU: {count, plural, =0 {No students} one {# student} other {# students}}
- Dates/times/numbers/currency always via Intl formatters
- Locale switcher in user menu; persisted in cookie
- Translation missing key logged in dev (not in prod)
- Translation PRs require linguist approval
- Automated check in CI: no hardcoded user-facing strings

### 23.3 Translation File Example

Translation files JSON format मध्ये आहेत — namespaced by module. ICU MessageFormat plurals, gender, dates handle करते. Missing keys fallback locale (en-IN) पासून येतात — user ला कधीच raw key string दिसत नाही:

```
// messages/en-IN/student.json
{
  "student": {
    "title": "Students",
    "create": {
      "title": "Create Student",
      "success": "Student created successfully",
      "error": "Failed to create student"
    },
    "list": {
      "empty": "No students found",
      "count": "{count, plural, =0 {No students} one {# student} other {# students}}"

```

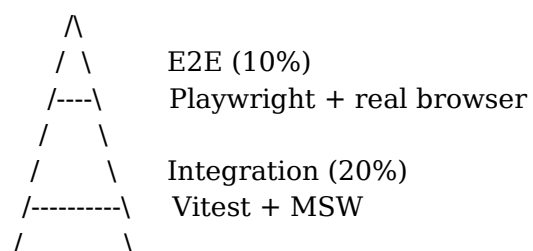
## 24. Testing

Testing हे frontend quality चा foundation आहे. PreOne चा testing approach classic test pyramid follow करतो — broad base of fast unit tests, narrow top of slow E2E tests. यामुळे feedback loop fast राहतो आणि confidence high राहतो. Accessibility testing + visual regression testing यांचा inclusion enterprise-grade discipline दर्शवतो.

## 24.1 Test Pyramid

| Layer               | Tool                                   | Scope                                           | Coverage Target          | Speed   |
|---------------------|----------------------------------------|-------------------------------------------------|--------------------------|---------|
| Unit Tests          | Vitest + jsdom                         | Pure functions, hooks, utils, validators        | 100% coverage target     | ms      |
| Component Tests     | Vitest + Testing Library               | Component rendering, interaction, accessibility | 85% coverage target      | ms      |
| Integration Tests   | Vitest + MSW (mock service worker)     | Hook + component + API mock end-to-end          | Critical paths           | seconds |
| E2E Tests           | Playwright                             | Real browser, full user journeys across routes  | Smoke + critical flows   | minutes |
| Accessibility Tests | axe-core + Vitest + Playwright         | WCAG violations auto-detected                   | Zero critical violations | seconds |
| Visual Regression   | Playwright screenshot diff + Chromatic | Pixel-level diff on component change            | Per Storybook story      | seconds |
| Performance Tests   | Lighthouse CI                          | Core Web Vitals per route                       | Per PR + weekly          | minutes |
| Contract Tests      | OpenAPI codegen + Vitest               | Frontend types match backend contract           | Per API change           | seconds |

## 24.2 Test Pyramid Diagram



/ \ Unit + Component (70%)  
 /-----\ Vitest + Testing Library

Speed: fast <-----> slow  
 Coverage: broad <-----> narrow  
 Cost: cheap <-----> expensive

## 24.3 Testing Rules

- 70% unit, 20% integration, 10% E2E — classic pyramid
- Component tests use Testing Library (query by role, not test-id)
- MSW for API mocking — never mock fetch directly
- E2E tests run against staging with seeded data
- Visual regression baseline requires designer approval to update
- Accessibility tests run on every PR — block merge on violations
- Lighthouse CI budget: 90+ on all metrics
- Tests run in parallel via Vitest + Playwright sharding
- Flaky tests auto-disabled after 3 flakes; ticket created
- Coverage reported in CI; PRs cannot decrease coverage

## 24.4 Component Test Example

Component test चा example खालीलप्रमाणे आहे. Testing Library use करते — query by role (not test-id), user-centric assertions. MSW API mocking करते — real HTTP semantics. Accessibility check axe-core सह automatic:

```
// modules/student/components/student-card.test.tsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { axe } from 'vitest-axe';
import { StudentCard } from './student-card';

describe('StudentCard', () => {
  const mockStudent: Student = {
    id: 'abc-123',
    firstName: 'Aarav',
    lastName: 'Sharma',
    className: 'Nursery-A',
    attendancePercent: 95,
    feeStatus: 'PAID',
  };
});
```

```

it('renders student name and class', () => {
  render(<StudentCard student={mockStudent} />);
  expect(screen.getByRole('heading', { name: /aarav
sharma/i })).toBeInTheDocument();
  expect(screen.getByText(/nursery-a/i)).toBeInTheDocument();
});

it('navigates to detail on click', async () => {
  const user = userEvent.setup();
  const onNavigate = vi.fn();
  render(<StudentCard student={mockStudent}
onNavigate={onNavigate} />);
  await user.click(screen.getByRole('button', { name: /aarav sharma/i }));
  expect(onNavigate).toHaveBeenCalledWith('abc-123');
});

it('has no accessibility violations', async () => {
  const { container } = render(<StudentCard student={mockStudent} />);
  const results = await axe(container);
  expect(results).toHaveNoViolations();
});
});

```

## 25. Deployment

Deployment हे production release चा final gate आहे — PreOne च्या deployment checklist ला strict enforcement आहे. यातील कोणताही item pending असेल तर deployment block होतो. PreOne frontend Vercel वर hosted आहे — edge network, instant rollbacks, preview deployments per PR.

### 25.1 Pre-Deployment Checklist

| Aspect     | Item                                      | Tool                |
|------------|-------------------------------------------|---------------------|
| Code       | All PRs merged; feature flags configured  | Git + Flagsmith     |
| Build      | next build passes; bundle within budget   | next build          |
| Tests      | Unit + integration + E2E + a11y all green | Vitest + Playwright |
| Lighthouse | All routes $\geq 90$ on all metrics       | Lighthouse CI       |
| Type Check | tsc --noEmit passes; no type errors       | tsc                 |

| Aspect            | Item                                            | Tool                    |
|-------------------|-------------------------------------------------|-------------------------|
| Lint              | ESLint + Prettier clean; 0 warnings             | ESLint                  |
| Env Vars          | All NEXT_PUBLIC_* set in Vercel env             | Vercel project settings |
| API Contract      | Backend deployed + contract verified            | OpenAPI diff            |
| Translations      | All keys translated for supported locales       | Crowdin sync            |
| Analytics         | Tracking events verified in staging             | PostHog / GA4           |
| Error Monitoring  | Sentry release tagged; source maps uploaded     | Sentry CLI              |
| Performance       | Bundle size within budget; no regressions       | Bundle analyzer         |
| Rollback Plan     | Instant rollback via Vercel (1-click)           | Vercel dashboard        |
| Smoke Tests       | Post-deploy smoke test (Playwright) on prod     | GitHub Actions          |
| Stakeholder Comms | Release notes circulated; stakeholders notified | Email / Slack           |

## 25.2 Deployment Topology

| Component          | Service                                             | Notes                           |
|--------------------|-----------------------------------------------------|---------------------------------|
| Hosting            | Vercel (primary) / self-hosted Node.js              | Edge network; instant rollbacks |
| CDN                | Vercel Edge Network (or CloudFront for self-hosted) | Global edge cache               |
| Image Optimization | Vercel Image Optimization (or next/image with S3)   | On-the-fly resize               |
| Analytics          | Vercel Analytics + PostHog (product analytics)      | Real user metrics               |
| Error Monitoring   | Sentry (browser + server)                           | Stack traces + source maps      |

| Component      | Service                                           | Notes                        |
|----------------|---------------------------------------------------|------------------------------|
| Session Replay | LogRocket (for critical flows only — sampling 1%) | UX debugging                 |
| Feature Flags  | Flagsmith (or Vercel Toolbar for dev)             | Progressive rollout          |
| A/B Testing    | Vercel Edge Config + Middleware                   | Server-side experiments      |
| i18n CDN       | Crowdin CDN for live translation updates          | No redeploy for translations |
| Fonts          | next/font (self-hosted via Vercel)                | No Google Fonts request      |

## 25.3 CI/CD Pipeline

CI/CD pipeline GitHub Actions वर आहे — प्रत्येक push चा build + test + a11y + Lighthouse scan होतो. PRs वर preview deployments auto-create होतात — reviewers URL शी live preview बघू शकतात. main branch merge झाल्यावर staging deploy होतो, production deploy manual approval नंतर होतो:

```

Push to feature branch
↓
CI: lint + typecheck + unit tests
↓ pass
CI: component tests + a11y tests
↓ pass
CI: build (next build) + bundle size check
↓ pass
CI: Lighthouse CI on critical routes
↓ pass (all metrics >= 90)
Vercel: preview deployment auto-created
↓
PR review with live preview URL
↓ approval + merge to main
CD: deploy to staging (auto)
↓
E2E tests on staging (Playwright)
↓ pass
Manual approval (release manager)
↓
CD: deploy to production (Vercel promotion)
↓
Smoke tests on production (Playwright smoke)
↓ pass
Release complete (1-click rollback available)

```

## 26. Coding Standards

Coding standards हे codebase consistency सुनिश्चित करतात — ज्यामुळे code read करणे, review करणे, आणि maintain करणे सोपे होते. PreOne चे standards strict पण pragmatic आहेत — developer productivity balance with code quality. ESLint + Prettier automated enforcement करतात; बाकी rules PR review मध्ये check होतात.

### 26.1 Standards Catalog

- Feature-first architecture — one module folder per business domain
- One module = one business domain (no cross-module business logic)
- Reusable UI components in components/ui/ — domain components in modules/
- No business logic inside components (only rendering + interaction)
- Hooks for reusable behavior — useStudent, usePermissions, useDebounce
- Strict TypeScript — no implicit any, no unchecked index access, no any
- ESLint + Prettier enforced via pre-commit hook (Husky)
- Import order: React → external → internal absolute → relative → types
- Named exports only (no default exports) — except page.tsx for Next.js
- File names: kebab-case (student-form.tsx, not StudentForm.tsx)
- Component names: PascalCase (StudentForm, not studentForm)
- Hook names: camelCase starting with 'use' (useStudent, not studentHook)
- Constant names: UPPER\_SNAKE\_CASE (MAX\_FILE\_SIZE, not maxFileSize)
- Type names: PascalCase (StudentProfile, not student\_profile)
- No magic numbers — extract to named constant
- No console.log in production code (use Sentry or proper logger)
- JSDoc on every public hook + utility
- Comments explain WHY, not WHAT

### 26.2 Pre-commit Hooks

Quality enforcement local level पर होण्यासाठी Husky pre-commit hooks install आहेत. प्रत्येक commit वर lint + format + type-check run होतात — failures commit block करतात. यामुळे broken code repository मध्ये push होत नाही:

```
// package.json
```



```

"husky": {
  "hooks": {
    "pre-commit": "lint-staged",
    "pre-push": "npm run test:unit"
  }
},
"lint-staged": {
  "*.{ts,tsx}": [
    "eslint --fix",
    "prettier --write",
    "tsc --noEmit"
  ]
}

```

## 26.3 PR Review Checklist

PR review मध्ये खालील checklist follow केला जातो. हे checklist reviewer ची cognitive load कमी करते — सर्व aspects systematically cover होतात:

- Components are pure — no API calls or global state access in components (use hooks).
- Server Components by default — 'use client' only when interactivity required.
- Zod schema exists for every form — single source of truth for validation.
- TanStack Query for server state — NEVER in Zustand.
- RBAC guards on all data-modifying buttons — <Can> component.
- Accessibility tested — axe-core passes, keyboard navigation works.
- Loading + error states implemented — not just happy path.
- Storybook story added for new components.
- Tests added — unit for logic, component for UI, E2E for critical flows.
- Bundle size within budget — no new heavy dependencies without ADR.
- Translations added for all user-facing strings (en-IN, hi, mr).
- Performance: Lighthouse score  $\geq 90$  on affected routes.

## 27. Folder Standards

Folder Standards हे codebase organization चा enforcement mechanism आहे — where files live, how they import each other, what goes where. PreOne मध्ये strict folder conventions आहेत जे ESLint import boundaries ने enforce होतात. Violation साठी lint error येतो आणि build fail होतो.

## 27.1 Folder Standards Catalog

- Each module folder has index.ts barrel exporting public API
- Modules never import from another module's internal files (only via index.ts)
- Shared components live in components/ui/ — not in modules/
- Shared hooks live in hooks/ — not in modules/
- Shared utilities live in utils/ — not in modules/
- Module-local types live in modules/{module}/types/ — not in root types/
- Module-local utilities live in modules/{module}/utils/ — not in root utils/
- Test files co-located with source: student-form.tsx → student-form.test.tsx
- Story files co-located: student-card.tsx → student-card.stories.tsx
- App Router routes in app/ — thin wrappers that compose module components
- No business logic in app/ routes — only composition + data fetching
- Layouts in layouts/ — reusable across routes
- Providers in providers/ — context wrappers, one per concern
- Services in services/ — API clients, one file per domain
- Config in lib/ — framework glue (queryClient, axiosClient, etc.)

## 27.2 ESLint Import Boundaries

Import boundary rules ESLint config मध्ये defined आहेत. खालील snippet rules चा essence दाखवते — full config repo मध्ये .eslintrc.js मध्ये आहे. हे rules ensure करतात की modules एकमेकांशी only via public barrels communicate करतात:

```
// .eslintrc.js
'import/no-restricted-paths': ['error', {
  zones: [
    // Modules cannot import from another module's internal files
    {
      target: './src/modules/**/!(*index.ts)',
      from: './src/modules/**/!(*index.ts)',
    },
    // Components cannot import from modules (use modules' public API)
    {
      target: './src/components',
      from: './src/modules',
    },
    // App router routes cannot contain business logic
    {
      target: './src/app',
```

```

    from: './src/modules/*/services',
  },
  {
    target: './src/app',
    from: './src/modules/*/api',
  },
],
}],
'import/no-internal-modules': ['error', {
  allow: ['@/modules/*', '@/components/ui/*', '@/hooks/*'],
}],

```

## 28. Glossary

Frontend terminology ची authoritative reference. नवीन terms introduce करताना या glossary मध्ये entry add करावी लागेल. Onboarding engineers साठी हा glossary पहिला read करायचा resource आहे — frontend vocabulary build करण्यासाठी.

| Term                 | Meaning                                                                          |
|----------------------|----------------------------------------------------------------------------------|
| App Router           | Next.js 13+ routing system using filesystem in app/ directory                    |
| RSC                  | React Server Components — components that render on server, zero JS shipped      |
| Server Actions       | Functions that run on server, called from client — no API endpoint needed        |
| Edge Runtime         | V8 isolate runtime for low-latency global execution (middleware, edge functions) |
| ISR                  | Incremental Static Regeneration — static pages re-generated at runtime           |
| Partial Prerendering | Next.js 14+ feature — static shell + streamed dynamic content                    |
| TanStack Query       | Server state library for data fetching, caching, mutations                       |
| Zustand              | Minimal client state library — no provider boilerplate                           |
| React Hook Form      | Performance-optimized form library using uncontrolled refs                       |
| Zod                  | TypeScript-first schema validation library                                       |

| Term             | Meaning                                                    |
|------------------|------------------------------------------------------------|
| ShadCN UI        | Component collection copied into repo (not npm dependency) |
| Radix UI         | Headless accessible primitives underlying ShadCN           |
| TanStack Table   | Headless table library for building data tables            |
| TanStack Virtual | Headless virtualization for long lists                     |
| Storybook        | Component development environment + living documentation   |

*Glossary (cont.)*

| Term            | Meaning                                                      |
|-----------------|--------------------------------------------------------------|
| MSW             | Mock Service Worker — API mocking for tests                  |
| Playwright      | Cross-browser E2E testing framework                          |
| Lighthouse      | Google's performance + accessibility auditing tool           |
| Core Web Vitals | Google's user experience metrics: LCP, INP, CLS              |
| WCAG            | Web Content Accessibility Guidelines — W3C standard          |
| RBAC            | Role-Based Access Control — permissions via roles            |
| ABAC            | Attribute-Based Access Control — permissions via attributes  |
| ACL             | Access Control List — per-resource permission list           |
| SSR             | Server-Side Rendering — HTML generated on server per request |
| SSG             | Static Site Generation — HTML generated at build time        |
| CSR             | Client-Side Rendering — HTML generated in browser via JS     |
| Hydration       | Attaching event listeners to server-rendered HTML            |

| Term             | Meaning                                        |
|------------------|------------------------------------------------|
| Suspense         | React component for declarative loading states |
| Error Boundary   | React component catching errors in tree below  |
| Provider Pattern | Context providers wrapping app to share state  |

## 29. Document Control

### 29.1 Version History

| Version | Date       | Author                            | Changes                                        |
|---------|------------|-----------------------------------|------------------------------------------------|
| 1.0     | 2026-07-14 | PreOne Frontend Architecture Team | Initial version — Frontend Architecture Freeze |

### 29.2 Approval Matrix

या document च्या freeze साठी खालील roles ची approval mandatory आहे. सर्व approvals recorded असल्यावरच document 'Architecture Freeze' status मध्ये move होतो:

| Role                  | Name  | Responsibility                         | Status  |
|-----------------------|-------|----------------------------------------|---------|
| Chief Architect       | [TBD] | Architecture review + sign-off         | Pending |
| VP Engineering        | [TBD] | Engineering approval                   | Pending |
| Frontend Lead         | [TBD] | Frontend architecture review           | Pending |
| UI/UX Lead            | [TBD] | Design system + theme alignment review | Pending |
| DevOps Lead           | [TBD] | Deployment + observability review      | Pending |
| Accessibility Officer | [TBD] | WCAG 2.2 AA compliance review          | Pending |

## 29.3 Review Cadence

- Architecture Freeze: quarterly review (Q1, Q2, Q3, Q4)
- Major version bump (v2.0): on framework upgrade (Next.js 17 → 18)
- Minor version bump (v1.1): on new chapter / significant section addition
- Patch version bump (v1.0.1): on clarification / typo fixes only

## 29.4 Distribution List

हा document खालील teams पर distributed आहे. Internal Engineering Reference — confidential, not for external distribution:

- PreOne Architecture Team (primary authors + maintainers)
- PreOne Frontend Engineering Team (all frontend engineers)
- PreOne Backend Engineering Team (API contract alignment)
- PreOne QA Team (test strategy + E2E reference)
- PreOne UI/UX Team (component library + theme reference)
- PreOne DevOps / SRE Team (deployment + monitoring reference)
- PreOne Onboarding Team (new hire orientation reference)
- PreOne Leadership (architectural investment decisions)

## 29.5 Companion Documents

हा document खालील documents सह read केला जावा — त्यांचा context this document ला supplement करतो:

| Document        | Version | Relationship                                        |
|-----------------|---------|-----------------------------------------------------|
| Vision Document | v1.0    | Strategic intent + non-functional requirements      |
| Master PRD      | v1.0    | Functional requirements — screen + flow mapping     |
| DDD             | v1.0    | Domain model — 14 modules alignment                 |
| ADR Catalog     | v1.0    | Architecture decisions — frontend choices rationale |

| Document                 | Version | Relationship                                    |
|--------------------------|---------|-------------------------------------------------|
| UI Design Philosophy     | v1.0    | Theme engine, component patterns, accessibility |
| API Contract Catalog     | v1.0    | Endpoint contracts — API services spec          |
| Backend Technical Design | v1.0    | Backend architecture — auth + RBAC alignment    |

## 29.6 Deliverables

या Frontend Architecture document मधून खालील production artifacts तयार होतील. हे artifacts implementation phase मध्ये frontend engineering team द्वारे deliver केले जातील:

- Complete Next.js Project Structure — apps/web/ with monorepo setup.
- 14 Business Domain Modules — feature-first organization with barrel exports.
- App Router Architecture — route groups, nested layouts, parallel routes.
- Global Layout System — 8 layouts covering all UI shells.
- State Management Strategy — TanStack Query + Zustand + React Hook Form separation.
- API Client Layer — 12 service files + axios interceptors + OpenAPI codegen.
- Authentication & Session Management — JWT + silent refresh + multi-school switch.
- RBAC Guards — 7 layers (route, menu, screen, component, button, field, API).
- Theme Engine & White Label Support — 5-level hierarchy + light/dark/auto modes.
- Form & Validation Framework — React Hook Form + Zod + 4-layer validation.
- Enterprise Component Library — 18 base + 10 composite components + Storybook.
- Dashboard & Data Table Standards — TanStack Table + 15 features + virtualization.
- Performance Optimization Guide — 12 tactics + SLOs + bundle budgets.
- Testing Strategy — 8 test layers including a11y + visual regression.
- Frontend Coding Standards — 18 rules + PR review checklist + ESLint boundaries.